



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático - Sistemas Distribuídos

Trabalho Prático de Sistemas Distribuídos: Parte 3

ARTHUR ATAÍDE DE MELO SARAIVA - 5070

MARIA EDUARDA DE PINHO BRAGA - 5099

GUILHERME BROEDEL ZORZAL - 5064

Sumário

1. Introdução.....	3
2. Organização.....	3
3. Desenvolvimento.....	5
3.1. Cliente.....	7
Responsabilidades.....	7
Funcionalidades, ou seja, funções definidas nesse módulo:.....	9
3.2. Interface.....	16
Estrutura principal.....	17
Estrutura geral.....	18
3.3. Servidor.....	22
Responsabilidades.....	23
server.py.....	23
database.py.....	26
Token e Criptografia.....	39
utils.py.....	41
objetos.py.....	43
handlers.....	45
4. Resultados.....	55
5. Conclusão.....	62
6. Referências.....	63

1. Introdução

O tema deste projeto é o desenvolvimento de um sistema distribuído utilizando tecnologias modernas e acessíveis. Dessa forma, através de um modelo de arquitetura cliente-servidor, foi possível estabelecer a comunicação entre as partes utilizando a API de Sockets com protocolo TCP. O objetivo desse desenvolvimento é oferecer uma solução robusta, modular e de fácil execução, que funcione adequadamente em ambientes com sistema operacional Ubuntu 21.04.

As técnicas utilizadas para a realização deste trabalho foram: programação em Python 3 com a utilização de sockets TCP para comunicação em rede, desenvolvimento de interface gráfica com PyQt6 e persistência de dados local com o banco de dados SQLite.

Em suma, o projeto demonstra a integração eficiente entre diferentes tecnologias para a construção de um sistema distribuído funcional e portátil, reforçando conceitos fundamentais da disciplina e promovendo a aplicação prática dos mesmos.

O trabalho foi desenvolvido usando a ferramenta Visual Studio Code, juntamente com a linguagem de programação Python e bibliotecas como a biblioteca socket, threading e json. O desenvolvimento foi realizado nos sistemas operacionais Linux Ubuntu e Linux Mint 22 Cinnamon.

2. Organização

Para garantir uma estrutura clara e modular, os arquivos do projeto foram organizados em diretórios conforme seu tipo e funcionalidade. Na **Figura 1**, é possível visualizar essa organização.

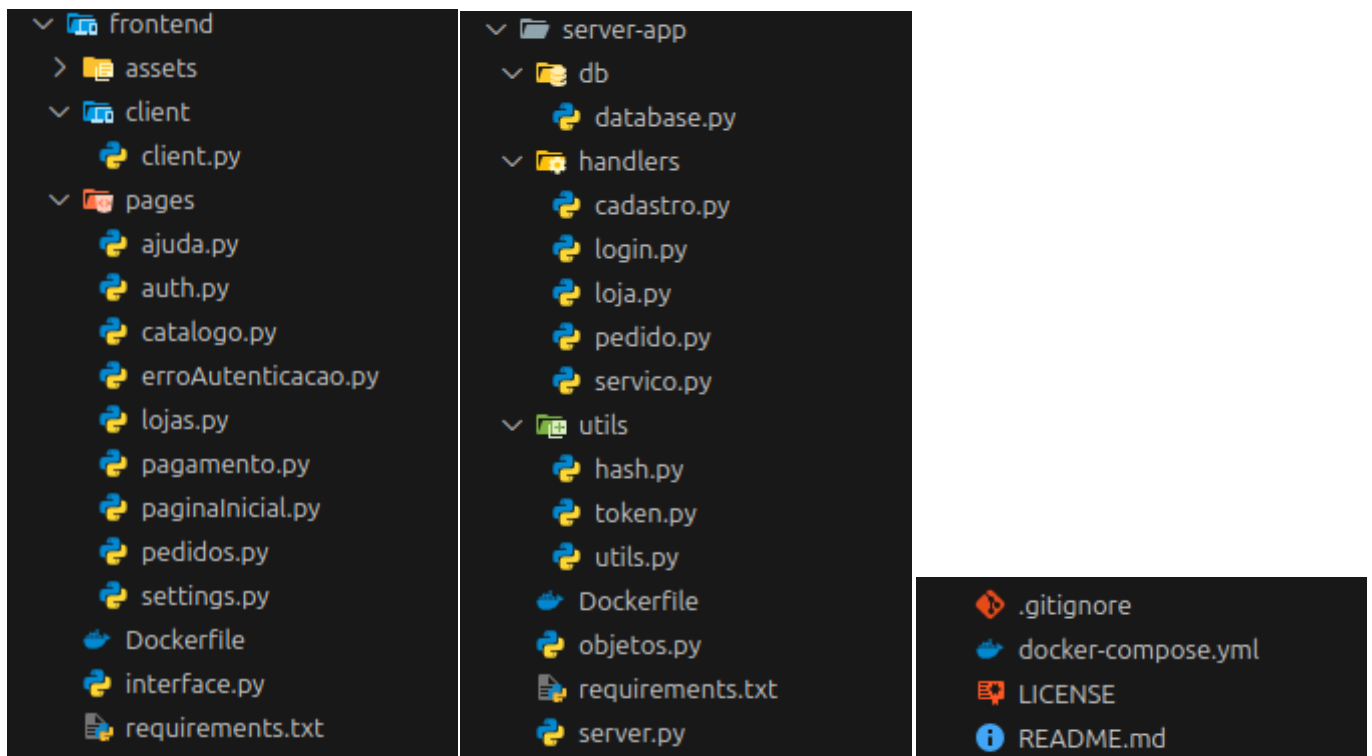


Figura 1 - Repositório do projeto.

A seguir, uma breve descrição de cada diretório e seus principais componentes:

1. Diretório raiz (. /):

- **docker-compose.yml:** Gerencia a execução dos containers Docker do sistema (cliente e servidor).
- **README.md:** Documentação principal e instruções de uso.

2. frontend/

- **assets/:** Imagens utilizadas na interface gráfica.
- **client/:** Contém client.py, responsável pela lógica principal do cliente.
- **Dockerfile:** Define o ambiente Docker para executar o cliente.
- **interface.py:** Código da interface gráfica principal, feito com PyQt6.
- **pages/:** Componentes de interface gráfica (login, catálogo, pedidos, etc).
- **requirements.txt:** Dependências Python do cliente.

3. server-app/

- **db/:** Contém database.py, responsável pelo gerenciamento do banco de dados SQLite.

- **handlers/**: Módulos que implementam as regras de negócio (cadastro, login, loja, pedido, serviço).
- **utils/**: Funções utilitárias como hash de senhas e geração de tokens.
- **objetos.py**: Define classes/estruturas de dados utilizadas no servidor.
- **server.py**: Código principal do servidor, gerencia conexões e encaminha requisições.
- **Dockerfile**: Define o ambiente Docker para executar o servidor.
- **requirements.txt**: Dependências Python do servidor.

O uso do Docker permite a execução simplificada através dos comandos `sudo docker compose up`, para versões mais recentes do Docker, ou, em caso de não funcionar, tente o comando `sudo docker-compose up`. Observação, para a primeira execução é necessário rodar `sudo docker compose up --build`, para baixar as bibliotecas necessárias (ou `sudo docker-compose up --build`).

Em caso de problema com as bibliotecas gráficas, que pode acontecer caso o Docker não tenha acesso ao sistema gráfico, rode os comandos de instalação separadamente, sendo eles: `sudo apt-get update && apt-get install -y libxcb-cursor0`.

3. Desenvolvimento

Para facilitar a configuração do ambiente e garantir a portabilidade do sistema, foi utilizado o **Docker**. O Docker permite a criação de containers que isolam as dependências do projeto, garantindo que o sistema funcione de forma correta em diferentes máquinas e ambientes.

Antes do desenvolvimento, fizemos a definição das funções, suas entradas e saídas, os objetos utilizados e como seria o formato das mensagens transmitidas para que pudéssemos integrar os componentes do sistema de forma correta.

Vale destacar antes de tudo o formato das mensagens trocadas, são elas:

Interface:

- Recebe as informações através de retornos das funções do cliente, no formato de um vetor de 3 posições. Veja na figura abaixo a ilustração da troca de informações cliente-interface:

```
return [resposta["status"], resposta["mensagem"], {}]
```

Figura 2 - Mensagem cliente-interface.

- Onde o primeiro elemento do vetor é o status da execução da função. O status é um código indicando o resultado da função, onde:
 - 200 representa sucesso,
 - 0 indica erro,
- O segundo elemento é a mensagem, uma string que descreve o status da execução, fornecendo informações adicionais ou explicações relacionadas ao código de status.
- E o terceiro elemento é o objeto de dados. Um dicionário contendo as informações que serão utilizadas pela interface.
- A interface envia informações ao cliente através dos parâmetros das funções invocadas.

Cliente:

- O Cliente troca mensagens com o servidor através da representação externas de dados JSON, seguindo o seguinte formato:

```
mensagem = {  
    "funcao": "<func_servidor>",  
    "dados": {  
        "key": valor,  
        "key2": 0  
    },  
}
```

Figura 2 - Mensagem cliente-servidor.

- Onde um objeto JSON mensagem é transmitido e esse objeto contém o nome da função a ser invocada no servidor e os dados que serão utilizados pela função invocada do servidor.

Servidor:

- O servidor então após o envio da mensagem de requisição do cliente, o retorna com um objeto JSON, no seguinte formato:

```
mensagem = {  
    "status": 200,  
    "mensagem": "Operacao feita com sucesso!",  
    "dados": {  
        "key": valor,  
        "key2": 0  
    },  
}
```

Figura 3 - Mensagem servidor-cliente.

- Status da execução, um código indicando o resultado da função, onde:
 - 200 representa sucesso,
 - 0 indica erro,
- Mensagem, uma string que descreve o status da execução, fornecendo informações adicionais ou explicações relacionadas ao código de status.
- Dados que serão utilizados pelo cliente.

3.1. Cliente

O módulo do Cliente foi definido na pasta `./frontend/client/client.py`. Toda a lógica de comunicação, autenticação, cadastro, operações de loja, catalogo, pedidos e anúncios passa pelo módulo do cliente.

A comunicação com o servidor é feita por meio de mensagens no formato JSON, enviadas via socket TCP. O cliente envia comandos e dados ao servidor, aguarda a resposta e processa o retorno para atualizar a interface e fornecer feedback ao usuário.

Responsabilidades

- Realizar o cadastro e autenticação de usuários.
- Permitir a criação e gerenciamento de lojas e anúncios.
- Consultar e retornar o catálogo de produtos/serviços disponíveis.
- Permite o Gerenciamento de pedidos: criação, visualização, pagamento, cancelamento.
- Mantém o controle do estado de autenticação do usuário (token).
- Garantir a comunicação segura e padronizada com o servidor.

Antes de definir as funcionalidades do cliente é necessário definir a função auxiliar utilizada por esse módulo para a troca de mensagens, a função `sendMessage`. Essa função é a responsável por permitir a troca de mensagens entre o cliente e o servidor, sua definição pode ser vista na Figura 4.

```
def sendMessage(host, port, mensagem):  
    """  
    Envia uma mensagem para o servidor e retorna a resposta.  
  
    :param host: Endereço do servidor  
    :param port: Porta do servidor  
    :param mensagem: Mensagem a ser enviada  
    :return: Resposta do servidor  
    """  
    print(f"Conectando ao servidor {host}:{port}...")  
  
    # Cria um socket para conexão TCP/IP  
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    # Conecta ao servidor no endereço e porta especificados  
    print(f"{mensagem}")  
    client.connect((host, port))  
  
    mensagem = json.dumps(mensagem)  
  
    # Envia os dados para o servidor  
    # O método encode() converte a string em bytes para envio  
    client.sendall(mensagem.encode())  
  
    response = client.recv(4096).decode()  
    client.close()  
  
    return json.loads(response)
```

Figura 4 - Função `sendMessage`.

Essa função usa o endereço `host` e a porta do servidor, passados como parâmetros, para criar um socket de conexão TCP/IP. Após a criação do socket, ele é usado para estabelecer a conexão com o servidor. Em seguida, a mensagem a ser enviada, também passada como parâmetro, é convertida para o formato JSON e, seguidamente, para bytes, sendo então enviada ao servidor. A resposta do servidor é recebida, convertida de bytes para string e, posteriormente, a conexão é encerrada. O retorno dessa função é a resposta do servidor, convertida para JSON.

A porta e o endereço `host` do servidor podem ser observados na Figura 5.

```
HOST = "server"  
PORT = 50051
```

Figura 5 - Porta e o endereço `host` do servidor.

Note que o docker cria um endereço virtual para cada container. Por isso acessamos o container com “server” e não “localhost”, por exemplo.

Funcionalidades, ou seja, funções definidas nesse módulo:

- **Cadastro de usuário:** Envia os dados de cadastro para o servidor e recebe um token de autenticação. Pode-se observar a função na Figura 6.

```
49 def cadastrar(nome, apelido, senha, ccm, contato):
50     """
51     Envia os dados de cadastro para o servidor.
52
53     :param nome: Nome da criatura mágica
54     :param apelido: Apelido da criatura mágica
55     :param senha: Senha para autenticação
56     :param ccm: Documento de identificação (Cadastro de Criatura Mágica)
57     :param contato: Informações de contato
58     :retorno: Resposta do servidor
59     """
60     try:
61         global tokenCliente
62
63         mensagem = {
64             "funcao": "cadastrar",
65             "dados": {
66                 "nome": nome,
67                 "apelido": apelido,
68                 "senha": senha,
69                 "ccm": ccm,
70                 "contato": contato,
71             },
72         }
73
74         # Aguarda e recebe a resposta do servidor
75         resposta = sendMessage(HOST, PORT, mensagem)
76
77         if resposta["status"] == 200:
78             tokenCliente = resposta["dados"].get("tokenCliente")
79
80         # Retorna a resposta do servidor
81         return [resposta["status"], resposta["mensagem"], {}]
82     except Exception as e:
83         print(f"Um erro ocorreu: {e} cadastrar")
84         return 0, e, {}
85
```

Figura 6 - Função para Cadastro de Usuários.

- **Login/autenticação:** Envia credenciais de login, ccm e senha, recebe token e carrega dados do usuário (loja, pedidos, catálogo) em paralelo usando threads. As threads foram usadas para otimizar o carregamento prévio dos dados dos clientes, como uma espécie de cache. Pode-se observar a função na Figura 7.

```

107 def autenticar(ccm, senha):
108     """
109     Envia os dados de autenticação para o servidor.
110
111     :param ccm: Documento de identificação (Cadastro de Criatura Mágica)
112     :param senha: Senha para autenticação
113     :return: Resposta do servidor
114     """
115     global tokenCliente
116     global data_possui_loja
117     global data_meus_pedidos
118     global data_pedidos_minha_loja
119     global data_catalogo
120
121     try:
122         mensagem = {"funcao": "autenticar", "dados": {"ccm": ccm, "senha": senha}}
123         resposta = sendMessage(HOST, PORT, mensagem)
124
125         if resposta["status"] == 200:
126             tokenCliente = resposta["dados"].get("tokenCliente")
127             print("Threads. Token:", tokenCliente)
128
129             results = {}
130
131             def run_and_store(name, func, *args, **kwargs):
132                 results[name] = func(*args, **kwargs)
133
134             thread1 = threading.Thread(
135                 target=run_and_store, args=("minha_loja", get_minha_loja)
136             )
137             thread2 = threading.Thread(
138                 target=run_and_store, args=("pedidos", get_pedidos)
139             )
140
141             thread3 = threading.Thread(
142                 target=run_and_store, args=("pedidos_minha_loja", get_pedidos_minha_loja),
143             )
144             thread4 = threading.Thread(
145                 target=run_and_store, args=("catalogo", get_catalogo)
146             )
147
148             thread1.start()
149             thread2.start()
150             thread3.start()
151             thread4.start()
152
153             thread1.join()
154             thread2.join()
155             thread3.join()
156             thread4.join()
157
158             data_possui_loja = results.get("minha_loja")
159             data_meus_pedidos = results.get("pedidos")
160             data_pedidos_minha_loja = results.get("pedidos_minha_loja")
161             data_catalogo = results.get("catalogo")
162
163             return [resposta["status"], resposta["mensagem"], {}]
164
165     except Exception as e:
166         print(f"Um erro ocorreu: {e} autenticar")
167         return 0, e, {}

```

Figura 7 - Função para Login/autenticação.

- **Criação de loja:** Permite ao usuário criar sua própria loja, enviando os dados necessários ao servidor, esses dados podem ser observados na definição da função presente na Figura 8.

```

174 def criar_loja(nome_loja, contato, descricao):
175     """
176     Envia os dados para criar uma loja no servidor.
177
178     :param nome_loja: Nome da loja
179     :param contato: Informações de contato da loja
180     :param descricao: Descrição da loja
181     :return: Resposta do servidor
182     """
183     try:
184         mensagem = {
185             "funcao": "criar_loja",
186             "dados": {
187                 "tokenCliente": tokenCliente,
188                 "nome": nome_loja,
189                 "contato": contato,
190                 "descricao": descricao,
191             },
192         }
193
194         # Resposta do servidor poderia ser o identificador da loja
195         resposta = sendMessage(HOST, PORT, mensagem)
196
197         return [resposta["status"], resposta["mensagem"], resposta["dados"]["loja"]]
198
199     except Exception as e:
200         print(f"Um erro ocorreu: {e} criar loja")
201         return 0, e, {}
202

```

Figura 8 - Função para Criação de Loja.

- **Criação e edição de anúncios:** Usuário pode criar (Figura 9), editar (Figura 10), ocultar (Figura 11), desocultar (Figura 12) e apagar anúncios (Figura 13) de serviços.
- **Consultar serviço:** Permitir a consulta das informações de um serviço através de seu identificador.

```

202 def criar_anuncio(nome, descricao, categoria, tipo, quantidade):
203     """ Cria um anúncio para um produto.
204     :param nome: Nome do produto
205     :param descricao: Descrição do produto
206     :param preco: Preço do produto
207     :param categoria: Categoria do produto
208     :return: Resposta do servidor
209     """
210     try:
211         mensagem = {
212             "funcao": "criar_anuncio",
213             "dados": {
214                 "tokenCliente": tokenCliente,
215                 "nome_servico": nome,
216                 "descricao_servico": descricao,
217                 "categoria": categoria,
218                 "tipo_pagamento": tipo,
219                 "quantidade": quantidade,
220             },
221         }
222
223         resposta = sendMessage(HOST, PORT, mensagem)
224
225         return [resposta["status"], resposta["mensagem"], {}]
226
227     except Exception as e:
228         print(f"Um erro ocorreu: {e} criar_anuncio")
229         return 0, e, {}
230

```

Figura 9 - Função para Criação do Anúncio do Serviço e do Serviço em si.

```

438 def editar_servico(idServico, nome, descricao, categoria, tipo, quantidade):
439     """
440     Edita um serviço existente.
441
442     :param idServico: ID do serviço
443     :param nome: Nome do produto
444     :param descricao: Descrição do produto
445     :param categoria: Categoria do produto
446     :param tipo: Tipo de pagamento
447     :param quantidade: Quantidade disponível
448     :return: Resposta do servidor
449     """
450     try:
451         mensagem = {
452             "funcao": "editar_servico",
453             "dados": {
454                 "idServico": idServico,
455                 "tokenCliente": tokenCliente,
456                 "nome_servico": nome,
457                 "descricao_servico": descricao,
458                 "categoria": categoria,
459                 "tipo_pagamento": tipo,
460                 "quantidade": quantidade,
461             },
462         }
463
464         resposta = sendMessage(HOST, PORT, mensagem)
465
466         return [resposta["status"], resposta["mensagem"], resposta["dados"]]
467     except Exception as e:
468         print(f"Um erro ocorreu: {e} editar_servico")
469         return 0, e, {}
470

```

Figura 10 - Função para Editar o Serviço.

```

472 def ocultar_servico(idServico):
473     """
474     Oculta um serviço específico.
475
476     :param idServico: ID do serviço
477     :return: Resposta do servidor
478     """
479     try:
480         mensagem = {
481             "funcao": "ocultar_servico",
482             "dados": {
483                 "idServico": idServico,
484                 "tokenCliente": tokenCliente,
485             },
486         }
487
488         resposta = sendMessage(HOST, PORT, mensagem)
489
490         return [resposta["status"], resposta["mensagem"], {}]
491
492     except Exception as e:
493         print(f"Um erro ocorreu: {e} ocultar_servico")
494         return 0, e, {}

```

Figura 11 - Função para Ocultar o Anúncio do Serviço.

```

497 def desocultar_servico(idServico):
498     """
499     Desoculta um serviço específico.
500
501     :param idServico: ID do serviço
502     :return: Resposta do servidor
503     """
504     try:
505         mensagem = {
506             "funcao": "desocultar_servico",
507             "dados": {
508                 "idServico": idServico,
509                 "tokenCliente": tokenCliente,
510             },
511         }
512         resposta = sendMessage(HOST, PORT, mensagem)
513         return [resposta["status"], resposta["mensagem"], {}]
514
515     except Exception as e:
516         print(f"Um erro ocorreu: {e} desocultar_servico")
517         return 0, e, {}
518
519

```

Figura 12 - Função para Desocultar o Anúncio do Serviço.

```

522 def apagar_servico(idServico):
523     """
524     Apaga um serviço específico.
525
526     :param idServico: ID do serviço
527     :return: Resposta do servidor
528     """
529     try:
530         mensagem = {
531             "funcao": "deletar_servico",
532             "dados": {
533                 "idServico": idServico,
534                 "tokenCliente": tokenCliente,
535             },
536         }
537         resposta = sendMessage(HOST, PORT, mensagem)
538         return [resposta["status"], resposta["mensagem"], {}]
539
540     except Exception as e:
541         print(f"Um erro ocorreu: {e} apagar_servico")
542         return 0, e, {}
543
544
545

```

Figura 13 - Função para Apagar o Serviço.

```

266 def get_servico(idServico):
267     """
268     Obtém os detalhes de um serviço específico.
269
270     :param idServico: ID do serviço
271     :return: Resposta do servidor
272     """
273     try:
274         mensagem = {
275             "funcao": "get_servico",
276             "dados": {"tokenCliente": tokenCliente, "idServico":
277                     idServico},
278         }
279         resposta = sendMessage(HOST, PORT, mensagem)
280         return [resposta["status"], resposta["mensagem"], resposta
281               [{"dados"]["servico"]}]]
282     except Exception as e:
283         print(f"Um erro ocorreu: {e} get_servico")
284         return 0, e, {}
285
286

```

Figura 14 - Função para consultar um serviço.

- **Consulta de catálogo:** Faz a consulta do catálogo de serviços para o servidor. Essa consulta pode ser filtrada por categoria ou loja, depende do parâmetro

passado durante a invocação da função. Pode-se observar a função na Figura 15.

```
249 def get_catalogo(categorias=[], idLoja=None):
250     """
251     Obtém o catálogo de produtos disponíveis.
252     :param categorias: Lista de categorias
253
254     :param idLoja: ID da loja
255     :return: Resposta do servidor
256     """
257     try:
258         mensagem = {
259             "funcao": "get_catalogo",
260             "dados": {
261                 "tokenCliente": tokenCliente,
262                 "pages": 0,
263                 "categorias": categorias,
264                 "idLoja": idLoja,
265             },
266         }
267         resposta = sendMessage(HOST, PORT, mensagem)
268
269         return [resposta["status"], resposta["mensagem"], resposta["dados"]["servicos"]]
270     except Exception as e:
271         print(f"Um erro ocorreu: {e} get_catalogo")
272         return 0, e, {}
273
```

Figura 15 - Função para consultar o catálogo.

- **Gestão de pedidos:** Usuário pode criar (Figura 16), consultar (Figura 17, pagar (Figura 18) e cancelar pedidos (Figura 19). Além disso, o usuário pode consultar pedidos feitos em sua loja (Figura 20).

```
548 def criar_pedido(idServico, quantidade):
549     """
550     Finalizar um pedido específico.
551
552     :param idPedido: ID do pedido
553     :return: Resposta do servidor
554     """
555     try:
556         mensagem = {
557             "funcao": "add_pedido",
558             "dados": {
559                 "idServico": idServico,
560                 "quantidade": quantidade,
561                 "tokenCliente": tokenCliente,
562             },
563         }
564
565         resposta = sendMessage(HOST, PORT, mensagem)
566
567         return [resposta["status"], resposta["mensagem"], {}]
568
569     except Exception as e:
570         print(f"Um erro ocorreu: {e} criar_pedido")
571         return 0, e, {}
572
```

Figura 16 - Função para criar um pedido.

```

335 def get_pedido(idPedido):
336     """
337     Obtém os detalhes de um pedido específico.
338
339     :param idPedido: ID do pedido
340     :return: Resposta do servidor
341     """
342     try:
343         mensagem = {
344             "funcao": "get_pedido",
345             "dados": {
346                 "idPedido": idPedido,
347                 "tokenCliente": tokenCliente,
348             },
349         }
350
351         resposta = sendMessage(HOST, PORT, mensagem)
352
353         return [resposta["status"], resposta["mensagem"], resposta["dados"]
354                 ["pedido"]]
355     except Exception as e:
356         print(f"Um erro ocorreu: {e} get_pedido")
357         return 0, e, {}

```

```

363 def get_pedidos():
364     """
365     Obtém a lista de pedidos.
366
367     :return: Resposta do servidor
368     """
369     try:
370         mensagem = {"funcao": "get_pedidos", "dados":
371                     {"tokenCliente": tokenCliente}}
372
373         resposta = sendMessage(HOST, PORT, mensagem)
374
375         return [resposta["status"], resposta["mensagem"],
376                 resposta["dados"]["pedidos"]]
377     except Exception as e:
378         print(f"Um erro ocorreu: {e} get_pedidos")
379         return 0, e, {}

```

Figura 17 - Função para consultar um pedido (à esquerda) e consultar todos os pedidos do cliente (à direita).

```

311 def pagar_pedido(idPedido):
312     """
313     Paga um pedido específico.
314
315     :param idPedido: ID do pedido
316     :return: Resposta do servidor
317     """
318     try:
319         mensagem = {
320             "funcao": "pagar_pedido",
321             "dados": {
322                 "idPedido": idPedido,
323                 "tokenCliente": tokenCliente,
324             },
325         }
326
327         resposta = sendMessage(HOST, PORT, mensagem)
328
329         return [resposta["status"], resposta["mensagem"], resposta["dados"]["pedido"]]
330     except Exception as e:
331         print(f"Um erro ocorreu: {e} pagar_pedido")
332         return 0, e, {}

```

Figura 18 - Função para pagar um pedido.

```

396 def cancelar_pedido(idPedido):
397     """
398     Cancela um pedido específico.
399
400     :param idPedido: ID do pedido
401     :return: Resposta do servidor
402     """
403     try:
404         mensagem = {
405             "funcao": "cancelar_pedido",
406             "dados": {
407                 "idPedido": idPedido,
408                 "tokenCliente": tokenCliente,
409             },
410         }
411
412         resposta = sendMessage(HOST, PORT, mensagem)
413
414         return [resposta["status"], resposta["mensagem"], {}]
415     except Exception as e:
416         print(f"Um erro ocorreu: {e} cancelar_pedido")
417         return 0, e, {}

```

Figura 19 - Função para cancelar um pedido.

```

376 def get_pedidos_minha_loja():
377     """
378     Obtém a lista de pedidos da loja do usuário.
379
380     :return: Resposta do servidor
381     """
382     try:
383         mensagem = {
384             "funcao": "get_pedidos_minha_loja",
385             "dados": {"tokenCliente": tokenCliente},
386         }
387
388         resposta = sendMessage(HOST, PORT, mensagem)
389
390         return [resposta["status"], resposta["mensagem"], resposta["dados"]["pedidos"]]
391     except Exception as e:
392         print(f"Um erro ocorreu: {e} get_pedidos_minha_loja")
393         return 0, e, {}
394

```

Figura 20 - Função para retornar os pedidos da loja do cliente.

Verificação de autenticação: Função para checar se o usuário está logado. Necessária para a autenticação o acesso às páginas dentro da interface. Pode-se observar a função na Figura 21.

```

150 def esta_logado():
151     """
152     Verifica se o usuário está logado.
153
154     :return: True se o usuário estiver logado, False caso contrário
155     """
156     if tokenCliente is not None:
157         return [200, "Usuário está autenticado!", {}]
158     else:
159         return [0, "Usuário não está autenticado!", {}]

```

Figura 21 - Função para verificar se o cliente está autenticado.

- **Logout:** Permite ao usuário encerrar sua sessão, invalidando o token localmente. Pode-se observar a função na Figura 22.

```

82 def logout():
83     global tokenCliente
84     tokenCliente = None
85     return [200, "Logout feito com sucesso", {}]

```

Figura 22 - Função para terminar a sessão do usuário.

- **Consultar loja:** Permitir a consulta das informações de uma loja através de seu identificador. Pode-se observar a função na Figura 23.


```

287 def get_loja(idLoja):
288     """
289     Obtém os detalhes de uma loja específica.
290
291     :param idLoja: ID da loja
292     :return: Resposta do servidor
293     """
294     try:
295         mensagem = {
296             "funcao": "get_loja",
297             "dados": {
298                 "idLoja": idLoja,
299                 "tokenCliente": tokenCliente,
300             },
301         }
302
303         resposta = sendMessage(HOST, PORT, mensagem)
304
305         return [resposta["status"], resposta["mensagem"], resposta["dados"]["loja"]]
306     except Exception as e:
307         print(f"Um erro ocorreu: {e} get_loja")
308         return 0, e, {}

```

Figura 23 - Função para consultar uma loja.

- **Consultar se usuário possui uma loja:** permite verificar que se o cliente já possui uma loja criada e associada a ele, isso é importante para a interface definir acesso a área “Loja”. Pode-se observar a função na Figura 24.

```

573 def usuario_possui_loja():
574     """
575     Verifica se o usuário possui uma loja.
576
577     :return: Resposta do servidor
578     """
579     try:
580         mensagem = {
581             "funcao": "tem_loja",
582             "dados": {"tokenCliente": tokenCliente},
583         }
584
585         resposta = sendMessage(HOST, PORT, mensagem)
586
587         return [resposta["status"], resposta["mensagem"], resposta["dados"]["resposta"]]
588
589     except Exception as e:
590         print(f"Um erro ocorreu: {e} usuario_possui_loja")
591         return 0, e, {}

```

Figura 24 - Função para verificar se o cliente já possui uma loja.

Observe que, nas imagens que mostram a definição da função, é possível identificar claramente as mensagens enviadas ao servidor (por meio do objeto mensagem presente em cada uma delas), bem como o tipo de retorno que a função envia à interface e os dados envolvidos na comunicação.

3.2. Interface

A interface gráfica está organizada no arquivo `./frontend/interface.py` e as páginas do sistema estão organizadas na pasta `./frontend/pages`.

Estrutura principal

Cada uma das classes principais do sistema é instanciada dentro de um `StackLayout` do `QtPy6` no arquivo “interface.py”. Dessa forma, o fluxo principal do sistema consiste na mudança da aba(objeto) que está sendo exibida no momento.

```
self.page_home = PaginaInicial(self)
self.page_auth = Auth(self)
self.page_catalogo = Catalogo(self)
self.page_lojas = Lojas(self)
self.page_pedidos = Pedidos(self)
self.page_ajuda = Ajuda(self)
self.page_erro = Erro(self)

self.pages = {
    "home": self.page_home,
    "auth": self.page_auth,
    "catalogo": self.page_catalogo,
    "loja": self.page_lojas,
    "pedidos": self.page_pedidos,
    "ajuda": self.page_ajuda,
    "erro": self.page_erro,
}

for page in self.pages.values():
    self.stack.addWidget(page)

# inicia na Home
self.stack.setCurrentWidget(self.page_home)

# Listas de paginas que requerem autenticao
self.protected_routes = {"catalogo", "loja", "pedidos"}
```

Figura 25 - Página da interface

Note que o `Widget` principal contém uma lista de rotas protegidas. A verificação da proteção das rotas é feita pela função abaixo: caso o usuário não esteja autenticado, ele é redirecionado para a página de erro. Essa verificação é feita durante as trocas de página, como pode ser conferido na função abaixo (Figura 26). A função de verificação de login “esta_logado” é uma das funções fornecidas pelo cliente e backend.

```
def navigate_to(self, page_name):
    print("LOGADO", esta_logado()[0])
    if page_name in self.protected_routes and not esta_logado()[0]:
        self.stack.setCurrentWidget(self.pages["erro"])
    else:
        if page_name == "loja":
            self.page_lojas.load()
        if page_name == "catalogo":
            self.page_catalogo.load()

        self.stack.setCurrentWidget(self.pages[page_name])
```

Figura 26 - Funcionamento do mecanismo de autenticação.

Os demais arquivos, cada qual com sua classe principal (PaginaInicial, Auth, Catalogo, Lojas, Pedidos, Ajuda, Erro), são responsáveis por gerenciar e organizar as suas respectivas páginas.

Estrutura geral

A implementação geral dessas páginas segue praticamente o mesmo princípio. No exemplo abaixo (FIGURA 27), temos a classe Pedidos, que possui uma estrutura muito semelhante às demais. A ideia principal de cada página consiste em:

- Inicializar e adicionar os pedidos à estrutura principal (Widget ou Layout).
- Escolher o Widget atual para as páginas multicomponentes.
- Montar e preparar as funções de mudanças de página, que lidam com o carregamento das páginas e outras funções que sejam necessárias
- Um ponto importante a se ressaltar é que no QtPy6, no momento em que a aplicação se inicia, todos os componentes são criados e, conseqüentemente, suas funções inits são executadas. Durante o desenvolvimento, isso acabou gerando diversos erros e atrasos devido ao timing do carregamento. Muitas dessas classes são carregadas com dados do usuário, porém os dados de um usuário requerem um token de acesso para serem recuperados e, portanto, as páginas exibiam erros de carregamento.
 - Para contornar esse problema, as classes em geral possuem uma função load, responsável por popular as classes com os dados quando chamada. Assim, a função é chamada apenas quando necessário e não gera erros inesperados.

```

class Pedidos(QStackedWidget):
    def __init__(self, parent):
        super().__init__(parent)
        self.meus_pedidos = MeusPedidos(self)
        self.pedidos_loja = PedidosLoja(self)
        self.area_pedidos = AreaPedidos(self)
        self.pedidos_unico = PedidoUnico(self)
        self.pedidos_unico_usuario = PedidoUnicoUsuario(self)

        self.addWidget(self.meus_pedidos)
        self.addWidget(self.pedidos_loja)
        self.addWidget(self.area_pedidos)
        self.addWidget(self.pedidos_unico)
        self.addWidget(self.pedidos_unico_usuario)

        self.setCurrentWidget(self.area_pedidos)

    def goto_meus_pedidos(self):
        self.meus_pedidos.load()
        self.setCurrentWidget(self.meus_pedidos)

    def goto_area_pedidos(self):
        self.setCurrentWidget(self.area_pedidos)

    def goto_pedido_unico(self, id):
        self.pedidos_unico.load(id)
        self.setCurrentWidget(self.pedidos_unico)

    def goto_pedido_unico_usuario(self, id):
        self.pedidos_unico_usuario.load(id)
        self.setCurrentWidget(self.pedidos_unico_usuario)

    def goto_pedidos_loja(self):
        self.pedidos_loja.load()
        self.setCurrentWidget(self.pedidos_loja)

```

Figura 27- Estrutura Geral de Páginas.

Com base nessas informações, percebe-se que as funções de load no geral são as mais importantes das classes da interface. Segue abaixo alguns exemplos importantes de classes que contém load:

```

class LojaStack(QStackedWidget):
    def __init__(self, parent): ...

    def load(self):
        if esta_logado()[0]:
            resp = usuario_possui_loja()
            if not resp[0]:
                QMessageBox.critical(self, "Erro", str(resp[1]))
                self.parent.goto_cria_loja()
                return
            possui = resp[2]
            if possui:
                self.area_loja.load()
                self.setCurrentWidget(self.area_loja)
            else:
                print("Nao possui loja")
                self.setCurrentWidget(self.cria_loja)
        else:
            QMessageBox.warning(
                self, "Erro de autenticação", "O usuario não está autenticado"
            )
            return

    def goto_area_loja(self):
        self.area_loja.load()
        self.setCurrentWidget(self.area_loja)

    def goto_cria_loja(self):
        self.setCurrentWidget(self.cria_loja)

```

Figura 28 - Base do componente de Loja.

```

class PedidosLoja(QWidget):
    def __init__(self, parent): ...

    def load(self):
        self.lista.clear()
        resp = get_pedidos_minha_loja()
        if not resp[0]:
            QMessageBox.warning(self, "Erro", str(resp[1]))
            return

        print("Pedido loja", resp[2])
        dados = resp[2]
        for dado in dados:
            pedido = Pedido(...)
            item = QListWidgetItem()

            item.setSizeHint(pedido.sizeHint())
            item.setData(Qt.ItemDataRole.UserRole, dado["idPedido"])

            self.lista.addItem(item)
            self.lista.setItemWidget(item, pedido)

    def goto_pedido_unico(self, id):
        self.parent.goto_pedido_unico_usuario(id)

    def voltar(self):
        self.parent.goto_area_pedidos()

```

Figura 29 - Base do componente de pedidos.

```

class CatalogoLista(QWidget):
    > def goto_servico(self, item):...
    > def load_services(self):...

    def load(self):
        self.lista.clear()
        self.current_page = 0
        try:
            resp = get_categoria()
            if not resp[0]:
                QMessageBox.warning(self, "Erro", str(resp[1]))
                return
            categoria = resp[2]

            self.filtro_categoria.addItem(categoria)
            resp = get_catalogo(page=self.current_page)
            print("Catalogo:", resp)
            if not resp[0]:
                QMessageBox.warning(self, "Erro", str(resp[1]))
                return

            services = resp[2]

            for service in services:
                item = QListWidgetItem()
                widget = CardServico(service)
                item.setSizeHint(widget.sizeHint())
                item.setData(Qt.ItemDataRole.UserRole, service["idServico"])
                self.lista.addItem(item)
                self.lista.setItemWidget(item, widget)

            if services:
                self.current_page += 1
            else:
                # Nada mais a carregar, desativa botão
                self.load_more_button.setEnabled(False)
                self.load_more_button.setText("Tudo carregado")

        except Exception as e:
            print(f"Erro ao carregar serviços: {e}")

```

Figura 30 - Base do componente de Catálogo

Para as demais telas e funcionalidades, a ideia é sempre a mesma. Realiza-se o carregamento dos dados “quando” e “onde” for necessário através das funções disponibilizadas pelo arquivo cliente (explicado anteriormente).

Por fim, um outro tipo de componente que foi montado para construção do frontend são classes que apenas guardam valores. Como essas classes são recriadas após certas requisições, não há necessidade de se ter funções “load” ou com lógicas mais complexas. São simplesmente classes templates de dados. Veja um exemplo desse tipo de classe na figura 31, responsável por guardar informações sobre os pedidos.

```

class Pedido(QWidget):
    def __init__(
        self, parent, id, data, servico, estado, total, nome_servico, nome_loja
    ):
        super().__init__(parent)
        self.parent = parent
        self.id = id
        self.title = QLabel("Pedido")
        self.title.setStyleSheet("""...
        self.data = QLabel(f"Data: {data}")
        self.estado = QLabel(f"Estado: {estado}")
        self.total = QLabel(f"Total: {total}")
        self.servico = QLabel(f"Nome do Servico: {nome_servico}")
        self.loja = QLabel(f"Loja: {nome_loja}")
        self.button_ver = QPushButton("Visualizar")
        self.button_ver.clicked.connect(self.visualizar)
        self.setStyleSheet("""...
        layout = QVBoxLayout()

        layout.addStretch()
        layout.addWidget(self.data)
        layout.addWidget(self.servico)
        layout.addWidget(self.estado)
        layout.addWidget(self.total)
        layout.addWidget(self.loja)
        layout.addWidget(self.button_ver)
        layout.addStretch()

        outer_layout = QHBoxLayout()
        outer_layout.addStretch()
        outer_layout.addLayout(layout)
        outer_layout.addStretch()
        self.setLayout(outer_layout)

    def visualizar(self):
        self.parent.goto_pedido_unico(self.id)

    def deletar(self):
        status, message = cancelar_pedido(self.id)
        if not status: ...
        else: ...

```

Figura 31 - Componente de Cadastro.

As telas resultantes da implementação do frontend podem ser conferidas na seção sobre os resultados do projeto.

3.3. Servidor

O módulo do servidor está localizado na pasta `./server-app/server.py` e é responsável por escutar conexões dos clientes, interpretar as mensagens recebidas, encaminhá-las ao manipulador correto e devolver uma resposta apropriada no formato JSON. Esse servidor foi desenvolvido utilizando a biblioteca socket para comunicação em rede, threading para lidar com múltiplas conexões simultâneas, e json para serialização das mensagens trocadas. O sistema também utiliza SQLite para persistência local dos dados.

Responsabilidades

- Escutar e aceitar conexões TCP dos clientes;
- Interpretar a mensagem recebida (em formato JSON) para identificar a função requisitada;
- Encaminhar a requisição para o módulo correspondente dentro da pasta handlers/;
- Garantir o tratamento de erros e resposta apropriada;
- Gerenciar sessões de usuário com tokens (armazenados temporariamente em memória);
- Acessar e manipular dados no banco de dados SQLite;
- Garantir concorrência no atendimento de múltiplas requisições com segurança e isolamento.

server.py

A função **main** é a responsável por iniciar e manter o servidor em funcionamento. Ela cria um **socket TCP**, configura o endereço (IP e porta) com o **bind** e começa a escutar conexões com o **listen**. Assim que um cliente se conecta, o servidor aceita a conexão (**accept**) e, imediatamente, cria uma nova **thread** dedicada à comunicação com aquele cliente, utilizando a função **handle_client**. O uso de threads aqui atende à exigência de concorrência do sistema distribuído: a thread principal do servidor fica responsável exclusivamente por aceitar conexões (receptor de requisições), enquanto cada nova conexão é processada em uma thread separada (processadora de requisições), garantindo que vários clientes possam ser atendidos ao mesmo tempo, de maneira independente. Isso evita que o servidor trave ou fique bloqueado ao atender apenas um cliente por vez. A implementação foi simples e eficaz, utilizando threading. As threads foram executadas com a flag **daemon=True**, o que permite encerrar as threads automaticamente ao fim do programa. Não foi necessário aplicar mecanismos explícitos de sincronização, como locks ou semáforos, já que o próprio sqlite lida com o gerenciamento de requisições de leitura e escrita ao banco de dados (vários usuários podem ler ao mesmo tempo, mas apenas um pode escrever). Devido a simplicidade da arquitetura de nosso sistema, com apenas dois containers/serviços, os casos onde mais de um usuário tenta escrever ao mesmo tempo não ocorrem. Porém, mesmo que ocorressem, o sqlite possui tratamento para isso e, caso não seja capaz de lidar com os múltiplos acessos de escritas, ele lança um erro que pode ser tratado no backend.


```

99  def main():
100      server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
101      server.bind((HOST, PORT))
102      server.listen()
103      print(f"Servidor ouvindo em {HOST}:{PORT}")
104      mostrar_tabelas()
105
106      while True:
107          conn, addr = server.accept()
108
109          # thread para processar cada conexão
110          t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
111          t.start()
112
113          # t.join()
114
115
116  if __name__ == "__main__":
117      main()
118

```

Figura 32 - Inicialização e Execução do Servidor.

Já a função **handle_client** é responsável por cuidar de cada cliente que se conecta ao servidor. Quando um cliente envia uma mensagem, essa função lê a requisição, tenta interpretá-la como um JSON e, se estiver tudo certo, encaminha para a função **tratar_mensagem**. Depois, ela formata a resposta em um padrão que o cliente entende e devolve essa resposta pela rede. Ela também fecha a conexão no final, garantindo que tudo fique limpo. Essa função é chamada toda vez que um novo cliente se conecta, o que faz com que o servidor consiga atender várias pessoas ao mesmo tempo de forma organizada.

```

82  def handle_client(conn, addr):
83      print(f"Conexão recebida de {addr}")
84      try:
85          req = conn.recv(1024)
86          if not req:
87              return
88          msg = json.loads(req.decode())
89      except json.JSONDecodeError:
90          conn.sendall(formatar_mensagem(0, "JSON inválido", {}).encode())
91          return
92      status, texto, dados = tratar_mensagem(msg)
93      resp = formatar_mensagem(status, texto, dados)
94      print(f"Resposta Servidor: {resp}")
95      conn.sendall(resp.encode())
96      conn.close()
97

```

Figura 33 - Processamento da Requisição do Cliente.

A função **tratar_mensagem** é o coração do servidor no que diz respeito ao processamento de requisições. Ela recebe um dicionário com a função desejada pelo cliente ("**funcao**") e os dados necessários para executá-la. Primeiro, ela verifica se a

requisição está no formato esperado. Em seguida, separa as funções que podem ser usadas sem autenticação (como **cadastro**, **login** e **visualizar as categorias**) daquelas que exigem um token válido para garantir a segurança. Se a função pedida for uma dessas mais sensíveis (como criar uma loja ou fazer um pedido), o servidor valida o token, identifica o cliente e repassa a solicitação para o respectivo módulo, como **loja**, **servico** ou **pedido**. Ela basicamente organiza a lógica do que o cliente está pedindo e direciona para o local certo, com ou sem verificação de segurança.

```
25 def tratar_mensagem(mensagem):
26     if "funcao" not in mensagem:
27         return formatar_mensagem(0, "Mensagem mal formatada", {})
28
29     func = mensagem["funcao"]
30     dados = mensagem.get("dados", {})
31
32     # Handlers que não precisam de autenticação
33     no_auth_handlers = {
34         "cadastrar": cadastro.cadastrar,
35         "autenticar": login.autenticar_cliente,
36         "get_categoria": lambda dados: servico.get_categoria(),
37     }
38
39     print(f"Dados do Cliente: ", func, dados)
40     if func in no_auth_handlers:
41         return no_auth_handlers[func](dados)
42
43     # Autenticação obrigatória
44     try:
45         status, msg, idCliente = autorizarToken(dados.get("tokenCliente", ""))
46         if status != 200:
47             return status, msg, {}
48     except Exception as e:
49         return 0, f"Erro ao verificar Token: {e}", {}
50
51     idCliente = int(idCliente)
52
53     # Handlers que requerem autenticação
54     auth_handlers = {
55         "criar_loja": lambda d: loja.criar_loja(d, idCliente),
56         "get_minha_loja": lambda d: loja.get_minha_loja(idCliente),
57         "tem_loja": lambda d: loja.tem_loja(idCliente),
58         "get_loja": lambda d: loja.get_loja(d),
59         "criar_anuncio": lambda d: servico.criar_anuncio(d, idCliente),
60         "get_catalogo": lambda d: servico.get_catalogo(d),
61         "get_servico": lambda d: servico.get_servico(d),
62         "ocultar_servico": lambda d: servico.mudar_estado_servico(d, 0),
63         "desocultar_servico": lambda d: servico.mudar_estado_servico(d, 1),
64         "deletar_servico": lambda d: servico.deletar_servico(d, idCliente),
65         "editar_servico": lambda d: servico.editar_servico(d, idCliente),
66         "add_pedido": lambda d: pedido.add_pedido(d, idCliente),
67         "pagar_pedido": lambda d: pedido.pagar_pedido(d, idCliente),
68         "get_pedido": lambda d: pedido.get_pedido(d, idCliente),
69         "get_pedidos": lambda d: pedido.get_pedidos(idCliente),
70         "get_pedidos_minha_loja": lambda d: pedido.get_pedidos_minha_loja(idCliente),
71         "cancelar_pedido": lambda d: pedido.cancelar_pedido(d, idCliente),
72         "reset": lambda d: (reset_database(), (200, "Banco de dados resetado", {}))[1],
73     }
74
75     handler = auth_handlers.get(func)
76     if not handler:
77         return formatar_mensagem(0, "Função não reconhecida", {})
78
79     return handler(dados)
```

Figura 34 - Roteamento e Autenticação de Requisições

database.py

Para facilitar os testes do sistema, um banco de dados com dados iniciais já foi disponibilizado. Caso esse banco não exista no computador no momento da execução, ele será criado automaticamente. O banco padrão inclui dois usuários pré-cadastrados, cada um com sua própria loja e serviços registrados. Isso permite testar diretamente as funcionalidades relacionadas a esses serviços de lojas, autenticação e manipulação de pedidos. Abaixo estão os dados de acesso para os testes:

- **Usuário 1** → CCM: 123 | Senha: 123
- **Usuário 2** → CCM: 456 | Senha: 456

A função **conectar** é essencial para garantir que a conexão com o banco de dados SQLite seja estabelecida corretamente. Ela também cria o diretório do banco (**/db**) se ele não existir.

```
6  FILE = "./db/sqlite.db"
7  FILEBASE = "./db/sqliteBase.db"
8
9
10 def conectar(file: str = FILE):
11     if not os.path.exists("./db"):
12         os.makedirs("./db")
13     return sqlite3.connect(file)
14
```

Figura 35 - Conectando ao banco de dados.

A função **mostrar_tabelas** permite visualizar todas as tabelas existentes no banco, além de listar seus dados atuais. Isso é útil para depuração ou para entender a estrutura atual do banco, a qual é utilizado apenas para testes, tendo essa melhor visualização dos dados.

```

15 def mostrar_tabelas():
16     conn = conectar()
17     cursor = conn.cursor()
18
19     cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
20     tabelas = cursor.fetchall()
21
22     print("Tabelas encontradas:")
23     for tabela in tabelas:
24         nome_tabela = tabela[0]
25         print(f"- {nome_tabela.upper()}")
26
27         # Exibe os dados de cada tabela
28         cursor.execute(f"SELECT * FROM {nome_tabela}")
29         linhas = cursor.fetchall()
30         print("Dados:")
31         for linha in linhas:
32             print(linha)
33         print("-" * 40)
34     print("-" * 40)
35     print("\n\n")
36     conn.close()
37

```

Figura 36 - Exibindo tabelas e seus dados.

A função **getClient** recuperar um cliente específico a partir do seu ID, montando um objeto **Cliente**.

```

39 def getClient(id):
40     con = conectar()
41     cur = con.cursor()
42     cur.execute("SELECT * FROM cliente WHERE idCliente = ?", (id,))
43     row = cur.fetchone()
44     con.close()
45     if row:
46         # Considerando a ordem: idCliente, nome, apelido, senha, ccm, contato
47         return Cliente(
48             idCliente=row[0],
49             nome=row[1],
50             apelido=row[2],
51             senha=row[3],
52             ccm=row[4],
53             contato=row[5],
54         )
55     return None
56

```

Figura 37 - Buscando cliente por ID

Já **addCliente** insere um novo cliente no banco e retorna seu ID, registrando seus dados como **nome**, **apelido**, **senha** e **contato**.

```

58 def addCliente(cliente: Cliente):
59     con = conectar()
60     cur = con.cursor()
61     try:
62         cur.execute(
63             "INSERT INTO cliente (nome, apelido, senha, ccm, contato) VALUES (?, ?, ?, ?, ?)",
64             (
65                 cliente.nome,
66                 cliente.apelido,
67                 cliente.senha,
68                 cliente.ccm,
69                 cliente.contato,
70             ),
71         )
72         con.commit()
73         cliente.idCliente = cur.lastrowid
74         return cliente.idCliente
75     except Exception as e:
76         print("Erro ao inserir cliente:", e)
77         con.rollback()
78         return None
79     finally:
80         con.close()
81

```

Figura 38 - Adicionando novo cliente

A **getLoja** busca informações de uma loja com base no ID da loja ou do cliente proprietário.

```

83 def getLoja(idCliente: int = None, idLoja: int = None):
84     con = conectar()
85     cur = con.cursor()
86
87     conditions = []
88     params = []
89     if idCliente is not None:
90         conditions.append("idCliente = ?")
91         params.append(idCliente)
92     if idLoja is not None:
93         conditions.append("idLoja = ?")
94         params.append(idLoja)
95
96     if not conditions:
97         con.close()
98         return None
99
100     # usa OR para procurar por cliente ou loja
101     sql = f"SELECT * FROM loja WHERE {' OR '.join(conditions)}"
102     cur.execute(sql, params)
103     row = cur.fetchone()
104     con.close()
105
106     if row:
107         return Loja(
108             idLoja=row[0],
109             nome=row[1],
110             contato=row[2],
111             descricao=row[3],
112             idCliente=row[4],
113         )
114     return None

```

Figura 39 - Buscando loja por ID ou cliente

A função **addLoja** insere uma nova loja vinculada a um cliente no banco. Ela registra nome, contato e descrição da loja.

```
117 def addLoja(loja: Loja):
118     con = conectar()
119     cur = con.cursor()
120     try:
121         cur.execute(
122             "INSERT INTO loja (nome, contato, descricao, idCliente) VALUES (?, ?, ?, ?)",
123             (loja.nome, loja.contato, loja.descricao, loja.idCliente),
124         )
125         con.commit()
126         loja.idLoja = cur.lastrowid
127         return loja.idLoja
128     except Exception as e:
129         print("Erro ao inserir loja:", e)
130         con.rollback()
131         return None
132     finally:
133         con.close()
```

Figura 40 - Adicionando nova loja

A **getService** retorna um serviço pelo ID, com opção de também retornar serviços apagados.

```
136 def getService(idServico, pegar_apagado=False):
137     con = conectar()
138     cur = con.cursor()
139     if pegar_apagado:
140         cur.execute("SELECT * FROM servico WHERE idServico = ?", (idServico,))
141     else:
142         cur.execute(
143             "SELECT * FROM servico WHERE idServico = ? AND apagado = 0", (idServico,)
144         )
145     row = cur.fetchone()
146     con.close()
147
148     if row:
149         return Servico(
150             idServico=row[0],
151             nome_servico=row[1],
152             descricao_servico=row[2],
153             categoria=row[3],
154             tipo_pagamento=row[4],
155             quantidade=row[5],
156             esta_visivel=bool(row[6]),
157             apagado=bool(row[7]),
158             idLoja=row[8],
159         )
160     return None
```

Figura 41 - Buscando serviço específico

A função **getServicos** é poderosa: busca vários serviços com filtros por loja,

categorias, visibilidade e paginação.

```
163 def getServicos(  
164     categorias: list[str] = None,  
165     idLoja: int = None,  
166     cont_pages: int = 0,  
167     page_size: int = 5,  
168     pegar_apagado: bool = False,  
169 ):  
170     con = conectar()  
171     cur = con.cursor()  
172  
173     conditions = []  
174     params = []  
175  
176     # só filtra apagados se não quisermos pegá-los  
177     if not pegar_apagado:  
178         conditions.append("apagado = 0")  
179  
180     # só filtra invisíveis se não quisermos pegá-los  
181     if not pegar_apagado and idLoja is None:  
182         conditions.append("esta_visivel = 1")  
183  
184     if idLoja is not None:  
185         conditions.append("idLoja = ?")  
186         params.append(idLoja)  
187  
188     if categorias and "todas" not in categorias:  
189         placeholders = ",".join("?" for _ in categorias)  
190         conditions.append(f"categoria IN ({placeholders})")  
191         params.extend(categorias)  
192  
193     where_clause = ""  
194     if conditions:  
195         where_clause = "WHERE " + " AND ".join(conditions)  
196  
197     offset = cont_pages * page_size  
198  
199     sql = f"""  
200     SELECT *  
201     FROM servico  
202     {where_clause}  
203     LIMIT ? OFFSET ?  
204     """
```

Figura 42 - Listando serviços com filtros parte 1

```

205     params.extend([page_size, offset])
206
207     cur.execute(sql, params)
208     rows = cur.fetchall()
209     con.close()
210
211     return [
212         Servico(
213             idServico=row[0],
214             nome_servico=row[1],
215             descricao_servico=row[2],
216             categoria=row[3],
217             tipo_pagamento=row[4],
218             quantidade=row[5],
219             esta_visivel=bool(row[6]),
220             apagado=bool(row[7]),
221             idLoja=row[8],
222         ).__dict__
223         for row in rows
224     ]

```

Figura 43 - Listando serviços com filtros parte 2

Com **mudarEstadoServico**, é possível ativar ou desativar a visibilidade de um serviço, para controle do que o cliente final vê.

```

227 def mudarEstadoServico(idServico, estado):
228     con = conectar()
229     cur = con.cursor()
230     try:
231         cur.execute(
232             "UPDATE servico SET esta_visivel = ? WHERE idServico = ?",
233             (1 if estado else 0, idServico),
234         )
235         con.commit()
236         return cur.rowcount > 0
237     except Exception as e:
238         print("Erro ao atualizar estado do serviço:", e)
239         con.rollback()
240         return False
241     finally:
242         con.close()
243

```

Figura 44 - Alterando visibilidade de serviço

A **addServico** registra um novo serviço (anúncio) em uma loja. É uma das bases do sistema, já que tudo gira em torno da venda desses serviços.


```

245 def addServico(servico: Servico):
246     con = conectar()
247     cur = con.cursor()
248     try:
249         nome_servico = getattr(servico, "nome_servico", "")
250         cur.execute(
251             "INSERT INTO servico (nome_servico, descricao_servico, categoria, tipo_pagamento, quantidade, esta_visivel, apagado, idLoja) VALUES"
252             "(?, ?, ?, ?, ?, ?, ?, ?)",
253             (
254                 nome_servico,
255                 servico.descricao_servico,
256                 servico.categoria,
257                 servico.tipo_pagamento,
258                 servico.quantidade,
259                 servico.esta_visivel,
260                 servico.apagado,
261                 servico.idLoja,
262             ),
263         )
264         con.commit()
265         servico.idServico = cur.lastrowid
266         return servico.idServico
267     except Exception as e:
268         print("Erro ao inserir serviço:", e)
269         con.rollback()
270         return None
271     finally:
272         con.close()

```

Figura 45 - Adicionando novo serviço

A função **editarServico** permite atualizar os dados principais de um serviço existente. Importante para manutenções ou correções.

```

274 def editarServico(servico: Servico):
275     con = conectar()
276     cur = con.cursor()
277     try:
278         cur.execute(
279             "UPDATE servico SET nome_servico = ?, descricao_servico = ?, categoria = ?, tipo_pagamento = ?, quantidade = ? WHERE idServico = ?",
280             (
281                 servico.nome_servico,
282                 servico.descricao_servico,
283                 servico.categoria,
284                 servico.tipo_pagamento,
285                 servico.quantidade,
286                 servico.idServico,
287             ),
288         )
289         con.commit()
290         return cur.rowcount > 0
291     except Exception as e:
292         print("Erro ao atualizar serviço:", e)
293         con.rollback()
294         return False
295     finally:
296         con.close()

```

Figura 46 - Editando dados de serviço

Com **delServico**, um serviço é marcado como apagado (soft delete), sendo removido da listagem pública sem ser excluído de fato, para os pedidos relacionados a eles serem mantidos sem causar erros ao código.

```

299 def delServico(idServico):
300     con = conectar()
301     cur = con.cursor()
302     try:
303         cur.execute(
304             "UPDATE servico SET apagado = 1 WHERE idServico = ?",
305             (idServico,),
306         )
307         con.commit()
308         return cur.rowcount > 0
309     except Exception as e:
310         print("Erro ao marcar serviço como apagado:", e)
311         con.rollback()
312         return False
313     finally:
314         con.close()
315

```

Figura 47 - Desativando serviço (apenas de forma lógica)

A **getPedido** retorna um pedido completo por ID, enriquecido com os nomes do cliente, serviço e loja, além de calcular o tempo de chegada.

```

317 def getPedido(idPedido):
318     con = conectar()
319     cur = con.cursor()
320     cur.execute("SELECT * FROM pedido WHERE idPedido = ?", (idPedido,))
321     row = cur.fetchone()
322     con.close()
323
324     if row:
325         return Pedido(
326             idPedido=row[0],
327             data_pedido=row[1],
328             data_pagamento=row[2],
329             data_entrega=row[3],
330             tempo_chegada=calcular_tempo_chegada(row[5], row[3]),
331             idServico=row[4],
332             estado_pedido=row[5],
333             total=row[6],
334             nome_cliente=getCliente(row[7]).nome,
335             nome_servico=getServico(row[4], pegar_apagado=True).nome_servico,
336             nome_loja=getLoja(
337                 idLoja=getServico(row[4], pegar_apagado=True).idLoja
338             ).nome,
339             idCliente=row[7],
340         )
341     return None
342

```

Figura 48 - Buscando pedido detalhado

A função **getPedidos** lista todos os pedidos feitos por um cliente, aplicando a verificação automática de conclusão com base na data de entrega.

```
344  def getPedidos(idCliente):
345      con = conectar()
346      cur = con.cursor()
347      cur.execute("SELECT * FROM pedido WHERE idCliente = ?", (idCliente,))
348      rows = cur.fetchall()
349      con.close()
350
351      pedidos = []
352      for row in rows:
353          pedido = Pedido(
354              idPedido=row[0],
355              data_pedido=row[1],
356              data_pagamento=row[2],
357              data_entrega=row[3],
358              tempo_chegada=calcular_tempo_chegada(row[5], row[3]),
359              idServico=row[4],
360              estado_pedido=row[5],
361              total=row[6],
362              nome_cliente=getCliente(row[7]).nome,
363              nome_servico=getServico(row[4], pegar_apagado=True).nome_servico,
364              nome_loja=getLoja(
365                  idLoja=getServico(row[4], pegar_apagado=True).idLoja
366              ).nome,
367              idCliente=row[7],
368          )
369          # se já passou da data de entrega, marca como concluído
370          verificar_entrega(pedido)
371          pedidos.append(pedido.__dict__)
372
373      return pedidos
```

Figura 49 - Listando pedidos da cliente

getPedidosLoja busca todos os pedidos feitos para a loja de um cliente, sendo ideal para o dono da loja visualizar suas vendas.

```

376 def getPedidosLoja(idCliente):
377     loja = getLoja(idCliente=idCliente)
378     if not loja:
379         return []
380
381     con = conectar()
382     cur = con.cursor()
383     sql = """
384     SELECT p.*
385     FROM pedido p
386     JOIN servico s ON p.idServico = s.idServico
387     WHERE s.idLoja = ?
388     """
389     cur.execute(sql, (loja.idLoja,))
390     rows = cur.fetchall()
391     con.close()
392
393     pedidos = []
394     for row in rows:
395         pedido = Pedido(
396             idPedido=row[0],
397             data_pedido=row[1],
398             data_pagamento=row[2],
399             data_entrega=row[3],
400             tempo_chegada=calcular_tempo_chegada(row[5], row[3]),
401             idServico=row[4],
402             estado_pedido=row[5],
403             total=row[6],
404             nome_cliente=getCliente(row[7]).nome,
405             nome_servico=getServico(row[4], pegar_apagado=True).nome_servico,
406             nome_loja=getLoja(
407                 idLoja=getServico(row[4], pegar_apagado=True).idLoja
408             ).nome,
409             idCliente=row[7],
410         )
411         # se já passou da data de entrega, marca como concluído
412         verificar_entrega(pedido)
413         pedidos.append(pedido.__dict__)
414
415     return pedidos
416

```

Figura 50 - Listando pedidos da loja

A **delPedido** remove um pedido do banco permanentemente, de forma física diferente do **delServico**.

```

418 def delPedido(idPedido):
419     con = conectar()
420     cur = con.cursor()
421     try:
422         cur.execute("DELETE FROM pedido WHERE idPedido = ?", (idPedido,))
423         con.commit()
424         return cur.rowcount > 0
425     except Exception as e:
426         print("Erro ao deletar pedido:", e)
427         con.rollback()
428         return False
429     finally:
430         con.close()
431

```

Figura 51 - Deletando pedido

A função **addPedido** insere um novo pedido no banco de dados. Ela recebe um objeto Pedido, extrai seus dados principais e realiza a inserção. Ao final, define o idPedido com o ID gerado automaticamente pela base.

```

433 def addPedido(pedido: Pedido):
434     con = conectar()
435     cur = con.cursor()
436     try:
437         cur.execute(
438             "INSERT INTO pedido (data_pedido, data_pagamento, data_entrega, idServico, estado_pedido, total, idCliente) VALUES (?, ?, ?, ?, ?, ?, ?)",
439             (
440                 pedido.data_pedido,
441                 pedido.data_pagamento,
442                 pedido.data_entrega,
443                 pedido.idServico,
444                 pedido.estado_pedido,
445                 pedido.total,
446                 pedido.idCliente,
447             ),
448         )
449         con.commit()
450         pedido.idPedido = cur.lastrowid
451         return pedido.idPedido
452     except Exception as e:
453         print("Erro ao inserir pedido:", e)
454         con.rollback()
455         return None
456     finally:
457         con.close()
458

```

Figura 52 - Registrar Novo Pedido

A função **atualizarDatasPedido** tem como objetivo atualizar duas datas importantes de um pedido: a de pagamento e a de entrega, quando o pedido for pago.

```

461 def atualizarDatasPedido(idPedido: int, data_pagamento: str, data_entrega: str) -> bool:
462     con = conectar()
463     cur = con.cursor()
464     try:
465         cur.execute(
466             "UPDATE pedido SET data_pagamento = ?, data_entrega = ? WHERE idPedido = ?",
467             (data_pagamento, data_entrega, idPedido),
468         )
469         con.commit()
470         return cur.rowcount > 0
471     except Exception as e:
472         print("Erro ao atualizar datas do pedido:", e)
473         con.rollback()
474         return False
475     finally:
476         con.close()

```

Figura 53 - Atualizar Data Do Pedido

mudarEstadoPedido recebe o ID de um pedido e um novo estado, e tenta atualizar o status no banco de dados — mas só se o estado for "ENVIADO" ou "CONCLUÍDO", garantindo que apenas transições válidas ocorram. É essencial para o controle do andamento dos pedidos no sistema. Se a atualização der certo, retorna True; senão, False.

```

479 def mudarEstadoPedido(idPedido, estado):
480     if estado != "ENVIADO" and estado != "CONCLUÍDO":
481         return False
482
483     con = conectar()
484     cur = con.cursor()
485     try:
486         cur.execute(
487             "UPDATE pedido SET estado_pedido = ? WHERE idPedido = ?", (estado, idPedido)
488         )
489         con.commit()
490         return cur.rowcount > 0
491     except Exception as e:
492         print("Erro ao atualizar status do pedido:", e)
493         con.rollback()
494         return False
495     finally:
496         con.close()

```

Figura 54 - Atualiza o estado de um pedido específico

Apaga o arquivo do banco de dados (se existir) e o recria do zero com a estrutura original. É uma função de suporte importante para testes ou quando se deseja começar com dados limpos. Serve como um "recomeçar do zero" no ambiente de desenvolvimento.

```

499 def reset_database():
500     if os.path.exists(FILE):
501         os.remove(FILE)
502
503     criar_banco()
504

```

Figura 55 - Reseta completamente o banco de dados

Responsável por montar as tabelas que sustentam o sistema: cliente, loja, serviço e pedido. Cada tabela é criada apenas se ainda não existir, e possui todos os relacionamentos e restrições necessárias. É a base da persistência de dados, e deve ser executada ao iniciar o sistema pela primeira vez ou após um reset.

```

507 def criar_banco():
508     con = conectar(FILEBASE)
509     cur = con.cursor()
510
511     # Tabela Cliente
512     cur.execute("""
513     CREATE TABLE IF NOT EXISTS cliente (
514         idCliente INTEGER PRIMARY KEY AUTOINCREMENT,
515         nome TEXT NOT NULL,
516         apelido TEXT NOT NULL,
517         senha TEXT NOT NULL,
518         ccm TEXT UNIQUE NOT NULL,
519         contato TEXT NOT NULL
520     );
521     """)
522
523     # Tabela Loja
524     cur.execute("""
525     CREATE TABLE IF NOT EXISTS loja (
526         idLoja INTEGER PRIMARY KEY AUTOINCREMENT,
527         nome TEXT NOT NULL,
528         contato TEXT NOT NULL,
529         descricao TEXT NOT NULL,
530         idCliente INTEGER NOT NULL,
531         FOREIGN KEY (idCliente) REFERENCES cliente(idCliente)
532     );
533     """)
534
535     # Tabela Serviço
536     cur.execute("""
537     CREATE TABLE IF NOT EXISTS servico (
538         idServico INTEGER PRIMARY KEY AUTOINCREMENT,
539         nome_servico TEXT NOT NULL,
540         descricao_servico TEXT NOT NULL,
541         categoria TEXT NOT NULL,
542         tipo_pagamento TEXT NOT NULL,
543         quantidade REAL NOT NULL,
544         esta_visivel BOOLEAN NOT NULL,
545         apagado BOOLEAN NOT NULL,
546         idLoja INTEGER NOT NULL,
547         FOREIGN KEY (idLoja) REFERENCES loja(idLoja)
548     );
549     """)

```

Figura 56 - Cria toda a estrutura do banco de dados parte 1

```

550     # Tabela Pedido
551     cur.execute("""
552     CREATE TABLE IF NOT EXISTS pedido (
553         idPedido INTEGER PRIMARY KEY AUTOINCREMENT,
554         data_pedido TEXT NOT NULL,
555         data_pagamento TEXT NOT NULL,
556         data_entrega TEXT NOT NULL,
557         idServico INTEGER NOT NULL,
558         estado_pedido TEXT NOT NULL,
559         total REAL NOT NULL,
560         idCliente INTEGER NOT NULL,
561         FOREIGN KEY (idServico) REFERENCES servico(idServico),
562         FOREIGN KEY (idCliente) REFERENCES cliente(idCliente)
563     );
564     """)
565
566     con.commit()
567     con.close()
568

```

Figura 57 - Cria toda a estrutura do banco de dados parte 2

Token e Criptografia

Em relação à segurança do sistema, foi utilizado um sistema de criptografia e autenticação por tokens JWT. A ideia é dar ao usuário um token de autenticação que é enviado em todas as requisições, e que impede boa parte de problemas de segurança comuns (funciona como um crachá para entrada de eventos. Só pode entrar se estiver autorizado). **hash_password** transforma a senha original do usuário em um hash usando o algoritmo Argon2 de criptografia, e **verify_password** recebe o hash da senha armazenado e a senha digitada pelo usuário, e verifica se elas correspondem (o banco de dados não guarda a senha do usuário em si, e sim sua versão criptografada).

```

You, há 5 dias | 1 author (You)
1  from argon2 import PasswordHasher
2  from argon2.exceptions import VerifyMismatchError
3
4  ph = PasswordHasher()
5
6  def hash_password(password):
7      return ph.hash(password)
8
9  def verify_password(stored_hash, provided_password):
10     try:
11         return ph.verify(stored_hash, provided_password)
12     except VerifyMismatchError:
13         return False
14
You, há 5 dias • tudo

```


Figura 58 - Criptografia

As funções relacionadas ao token são responsáveis por gerar, validar, decodificar e autorizar tokens JWT de forma segura, garantindo que apenas clientes autenticados possam acessar rotas protegidas e que cada requisição seja devidamente associada ao cliente correto.

```
9
10 def gerar_token(cliente):
11     now = datetime.datetime.now(BR)
12
13     payload = {
14         "sub": str(cliente.idCliente),
15         "iat": int(now.timestamp()),
16         "nbf": int(now.timestamp()),
17         "exp": int((now + EXP_DELTA).timestamp()),
18         "apelido": cliente.apelido,
19     }
20     token = jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
21     return token
22
```

Figura 59 - Token parte 1

```
23 def decode_token(token):
24
25     if isinstance(token, (bytes, bytearray)):
26         token = token.decode("utf-8")
27     token = token.strip()
28     token = token.strip('"\'')
29     if not token:
30         return 0, "Token não fornecido", {}
31
32     try:
33         payload = jwt.decode(
34             token,
35             SECRET_KEY,
36             algorithms=[ALGORITHM],
37             options={
38                 "require": ["exp", "iat", "sub"],
39                 "verify_exp": True
40             },
41             leeway=10 # tolerância de 10s para clock skew
42         )
43         return 200, "Token válido", payload
44
45     except jwt.ExpiredSignatureError:
46         return 0, "Token expirado. Faça login novamente.", {}
47     except jwt.MissingRequiredClaimError as e:
48         return 0, f"Claim ausente: {e.claim}", {}
49     except jwt.InvalidTokenError as error:
50         return 0, "Token inválido. Verifique suas credenciais.", {}
```

Figura 60 - Token parte 2

```

52  def protected(token):
53      status, msg, payload = decode_token(token)
54      return status, msg, payload if status == 200 else {}
55
56  def autorizarToken(token):
57      status, msg, payload = protected(token)
58
59      if status != 200:
60          return status, msg, {}
61      return 200, "ID do cliente obtido", payload.get("sub")
62
63  def authorization(idCliente, token):
64
65      cliente = getCliente(idCliente)
66      if cliente is None:
67          return 0, "Usuário não encontrado", {}
68
69      status, msg, payload = protected(token)
70      if status != 200:
71          return status, msg, {}
72
73      if payload.get("sub") != int(idCliente):
74          return 0, "Usuário não autorizado", {}
75
76      return 200, "Usuário autorizado", {"cliente": cliente}

```

Figura 61 - Token parte 3

utils.py

A função **formatar_mensagem** organiza os dados do pedido e os formata antes de enviá-los como resposta para o cliente. Ela é um ponto central para manter consistência na API, como tinha sido comentado anteriormente com o json {"status": status, "mensagem": mensagem, "dados": dados}. Já **formatar_pedido** ajusta os dados de um ou mais pedidos, preparando-os para visualização. Ele chama internamente **formatar_data**, padronizando datas para uma melhor visualização, a qual converte datas ISO para o formato brasileiro e ajusta a exibição do tempo estimado de chegada, levando em conta o estado atual do pedido.

```

10  def formatar_mensagem(status, mensagem, dados):
11      dados["pedido"] = formatar_pedido(dados)
12
13      return json.dumps({"status": status, "mensagem": mensagem, "dados": dados})
14
15
16  def formatar_pedido(dados):
17      if dados.get("pedido") is None and dados.get("pedidos") is None:
18          return False
19
20      if dados.get("pedido") is not None:
21          formatar_data(dados["pedido"])
22          return dados["pedido"]
23
24      if dados.get("pedidos") is not None:
25          for pedido in dados["pedidos"]:
26              formatar_data(pedido)
27      return dados["pedidos"]
28
29      return False
30

```

Figura 62 - Formatação de padronização parte 1

```

32  def formatar_data(pedido: Pedido):
33      # formato = dia / mes / ano - hora : minuto : segundo
34      formato = "%d/%m/%Y - %H:%M:%S"
35
36      pedido["data_pedido"] = datetime.datetime.fromisoformat(
37          pedido["data_pedido"]
38      ).strftime(formato)
39
40      if pedido["tempo_chegada"] != "Esperando pagamento":
41          # formato = hora : minuto : segundo
42          if ":" in pedido["tempo_chegada"] or "." in pedido["tempo_chegada"]:
43              hms, *_ = calcular_tempo_chegada(
44                  pedido["estado_pedido"],
45                  pedido["data_entrega"],
46              ).split(".")
47              if hms == "Esperando pagamento" or hms == "Pedido concluído":
48                  pedido["tempo_chegada"] = hms
49
50              else:
51                  h, m, s = hms.split(":")
52                  pedido["tempo_chegada"] = f"{int(h):02d}:{int(m):02d}:{int(s):02d}"
53
54      if pedido["data_pagamento"] != "Esperando pagamento":
55          pedido["data_pagamento"] = datetime.datetime.fromisoformat(pedido["data_pagamento"]).strftime(formato)
56
57      if pedido["data_entrega"] != "Esperando pagamento":
58          pedido["data_entrega"] = datetime.datetime.fromisoformat(pedido["data_entrega"]).strftime(formato)
59

```

Figura 63 - Formatação de padronização parte 2

A função **calcular_tempo_chegada** calcula o tempo restante para que um pedido chegue ao destino, com base no horário atual e na data de entrega. Por fim, **verificar_entrega** checa se um pedido enviado já ultrapassou sua data de entrega e, se sim, atualiza seu estado para "CONCLUIDO" (a ideia é simular que o pedido foi entregue pelo dono da loja).

```
60
61 # tempo de entrega - tempo atual
62 def calcular_tempo_chegada(estado_pedido, data_entrega):
63     if estado_pedido == "PENDENTE":
64         return "Esperando pagamento"
65
66     elif estado_pedido == "ENVIADO":
67         # aceita ISO ou formato já formatado (dd/mm/YYYY - HH:MM:SS)
68         try:
69             if "/" in data_entrega:
70                 entrega = datetime.datetime.strptime(
71                     data_entrega, "%d/%m/%Y - %H:%M:%S"
72                 )
73             else:
74                 entrega = datetime.datetime.fromisoformat(data_entrega)
75             delta = entrega - datetime.datetime.now(BR)
76             return str(delta)
77         except Exception:
78             return "Data de entrega inválida"
79     elif estado_pedido == "CONCLUIDO":
80         return "Pedido concluído"
81
82     return False
83
```

Figura 64 - Cálculo do Tempo de Chegada

```
85 def verificar_entrega(pedido: Pedido):
86     if pedido.estado_pedido == "ENVIADO":
87         entrega = datetime.datetime.fromisoformat(pedido.data_entrega)
88         if entrega < datetime.datetime.now(BR):
89             pedido.estado_pedido = "CONCLUIDO"
90             db.mudarEstadoPedido(int(pedido.idPedido), pedido.estado_pedido)
91             return True
92     return False
```

Figura 65 - Verificação Automática de Entrega Concluída

objetos.py

Este trecho define as estruturas principais do sistema por meio de data classes —

Cliente, Loja, Serviço e Pedido — representando, respectivamente, os usuários do sistema, suas lojas mágicas, os serviços ofertados e os pedidos realizados, além de listar as categorias disponíveis para serviços mágicos, organizando de forma clara e tipada os dados manipulados na aplicação.

```
4 # categorias de serviços magicos
5 v categorias = []
6     'todas',
7     'alquimia',
8     'assassinato',
9     'ataques',
10    'cura',
11    'espionagem',
12    'guarda-costas',
13    'magia',
14    'mineração',
15    'rituais',
16    'trabalho',
17    'transporte',
18    'outros'
19 ]
20
21 You, há 5 dias | 1 author (You)
22 @dataclass
23 v class Cliente:
24     idCliente: Optional[int] = None
25     nome: str = ""
26     apelido: str = ""
27     senha: str = ""
28     ccm: str = ""
29     contato: str = ""
30
31 You, há 5 dias | 1 author (You)
32 @dataclass
33 v class Loja:
34     idLoja: Optional[int] = None
35     nome: str = ""
36     contato: str = ""
37     descricao: str = ""
38     idCliente: int = 0
```

Figura 66 - Classes parte 1

```

40 @dataclass
41 class Servico:
42     idServico: Optional[int] = None
43     nome_servico: str = ""
44     descricao_servico: str = ""
45     categoria: str = ""
46     tipo_pagamento: str = ""
47     quantidade: float = 0.0
48     esta_visivel: bool = True
49     apagado: bool = False
50     idLoja: int = 0
51
52
53 You, ontem | 1 author (You)
54 @dataclass
55 class Pedido:
56     idPedido: Optional[int] = None
57     data_pedido: str = ""
58     data_pagamento: str = "Esperando pagamento"
59     data_entrega: str = "Esperando pagamento"
60     tempo_chegada: str = "Esperando pagamento"
61     idServico: int = 0
62     estado_pedido: str = "PENDENTE" # PENDENTE, ENVIADO, CONCLUÍDO
63     total: float = 0.0
64     nome_cliente: str = ""
65     nome_servico: str = ""
66     nome_loja: str = ""
67     idCliente: int = 0

```

Figura 67 - Classes parte 2

handlers

Essas duas funções lidam com os processos fundamentais de login e cadastro de clientes no sistema. A função **autenticar_cliente** valida as credenciais informadas (CCM e senha), verifica se a senha corresponde à armazenada e, em caso positivo, gera um token JWT de autenticação para acesso seguro. Já a função **cadastrar** cria um novo cliente no banco de dados a partir das informações fornecidas, realizando a criptografia segura da senha e retornando também um token, permitindo que o usuário esteja autenticado logo após o cadastro. Ambas são cruciais para garantir segurança, controle de acesso e integridade na autenticação de usuários.

```

You, há 8 horas | 1 author (You)
1  from utils.token import gerar_token
2  from db.database import conectar, getCliente
3  from utils.hash import verify_password
4
5  def autenticar_cliente(dados):
6      conn = conectar()
7      cur = conn.cursor()
8      try:
9          # busca hash e id pelo ccm
10         cur.execute(
11             "SELECT idCliente, senha FROM cliente WHERE ccm = ?",
12             (dados["ccm"],)
13         )
14         row = cur.fetchone()
15         if not row:
16             return 0, "CCM ou senha incorretos", {}
17
18         id_cliente, stored_hash = row
19         # verifica senha
20         if not verify_password(stored_hash, dados["senha"]):
21             return 0, "CCM ou senha incorretos", {}
22
23         cliente = getCliente(id_cliente)
24         if cliente is None:
25             return 0, "Erro: cliente não encontrado", {}
26
27         token = gerar_token(cliente)
28         return 200, "Autenticação realizada com sucesso", {"tokenCliente": token}
29
30     except Exception as e:
31         return 0, f"Erro ao autenticar: {e}", {}
32     finally:
33         conn.close()

```

Figura 68 - Validação de login com token JWT

```

1  from utils.token import gerar_token
2  from db.database import addCliente
3  from objetos import Cliente
4  from utils.hash import hash_password
5
6  def cadastrar(dados):
7      try:
8          # Cria a instância do cliente com os dados recebidos
9          cliente = Cliente(
10             nome=dados['nome'],
11             apelido=dados['apelido'],
12             senha=hash_password(dados['senha']),
13             ccm=dados['ccm'],
14             contato=dados['contato']
15         )
16
17         # Insere o cliente no banco e atualiza o id na instância
18         id_cliente = addCliente(cliente)
19         if id_cliente is None:
20             return 0, "Erro ao cadastrar: cliente não inserido", {}
21
22         token = gerar_token(cliente)
23
24         return 200, "Cadastro realizado com sucesso", {"tokenCliente": token}
25
26     except Exception as e:
27         return 0, f"Erro ao cadastrar: {str(e)}", {}
28

```

Figura 69 - Cadastro seguro de novo cliente

Essas funções gerenciam o ciclo de vida e a consulta das lojas vinculadas aos clientes no sistema. A **criar_loja** permite que um cliente crie sua loja, desde que ainda não possua uma. Ela cria uma instância da loja com os dados recebidos e a armazena no banco. A **get_minha_loja** permite que o cliente consulte sua loja atual, enquanto **tem_loja** verifica se ele já possui alguma loja vinculada, evitando duplicações. Por fim, a **get_loja** permite recuperar qualquer loja pelo seu idLoja.


```

1  from db.database import addLoja, getLoja
2  from objetos import Loja
3  from utils.token import autorizarToken
4
5  def criar_loja(dados, idCliente):
6      aux = tem_loja(idCliente)
7      if aux[0] == 0:
8          return 0, "Erro ao verificar loja", {}
9      if aux[2]['resposta']:
10         return 0, "Usuário já possui loja", {}
11
12     try:
13         loja = Loja(
14             nome=dados['nome'],
15             contato=dados['contato'],
16             descricao=dados['descricao'],
17             idCliente=idCliente
18         )
19         # You, há 5 dias • tudo
20         id_loja = addLoja(loja)
21
22         if id_loja is None:
23             return 0, "Erro ao criar a loja: não foi possível inserir no banco", {}
24
25         loja.idloja = id_loja
26
27         return 200, "Loja criada com sucesso", {"loja": loja.__dict__}
28
29     except Exception as e:
30         return 0, f"Erro ao criar loja: {str(e)}", {}
31

```

Figura 70 - Criação de loja para cliente

```

32 def get_minha_loja(idCliente):
33     try:
34         loja = getLoja(idCliente=idCliente)
35         if loja:
36             return 200, "Loja encontrada", {"loja": loja.__dict__}
37         else:
38             return 0, "Loja não encontrada", {}
39
40     except Exception as e:
41         return 0, f"Erro ao obter loja: {e}", {}
42
43 # faça verificar caso o usuario possui uma loja
44 def tem_loja(idCliente):
45
46     try:
47         loja = getLoja(idCliente=idCliente)
48         if loja:
49             return 200, "Usuário possui loja", {"resposta": True}
50         else:
51             return 200, "Usuário não possui loja", {"resposta": False}
52
53     except Exception as e:
54         return 0, f"Erro ao verificar loja: {e}", {}
55

```

Figura 71 - Consulta loja do cliente autenticado e Verificação de loja

```

56  def get_loja(dados):
57      try:
58          idLoja = dados['idLoja']
59          loja = getLoja(idLoja=idLoja)
60          if loja:
61              return 200, "Loja encontrada", {"loja": loja.__dict__}
62          else:
63              return 0, "Loja não encontrada", {}
64
65      except Exception as e:
66          return 0, f"Erro ao obter loja: {e}", {}
67

```

Figura 72 - Busca de loja por ID

As funções do módulo de pedidos possibilitam a interação entre clientes e lojas na compra de serviços mágicos. A **add_pedido** permite a criação de um pedido, verificando a disponibilidade do serviço e impedindo que o cliente compre de seu próprio estabelecimento. A **pagar_pedido** muda o estado do pedido para "**ENVIADO**", atualizando automaticamente as datas de pagamento e previsão de entrega. A função **get_pedido** permite que tanto o cliente quanto o vendedor consultem os detalhes de um pedido específico, enquanto **get_pedidos** e **get_pedidos_minha_loja** listam todos os pedidos feitos por um cliente e recebidos por uma loja, respectivamente. Por fim, **cancelar_pedido** permite ao cliente excluir um pedido, tenha ele sido pago ou não.

Figura 73 - Criação de pedido de serviço

```

43 def pagar_pedido(dados, idCliente):
44     pid = dados.get("idPedido")
45     pedido = db.getPedido(int(pid))
46
47     if not pedido:
48         return 0, "Pedido não encontrado", {}
49
50     if int(pedido.idCliente) != idCliente:
51         return 0, "Usuário não autorizado", {}
52
53     if pedido.estado_pedido != "PENDENTE":
54         return 0, "Pedido já pago", {}
55
56     pedido.estado_pedido = "ENVIADO"
57
58     if db.mudarEstadoPedido(int(pid), pedido.estado_pedido):
59         pago_em = datetime.datetime.now(BR).isoformat()
60         min = random.randint(2, 10)
61         previsto_para = (
62             datetime.datetime.now(BR) + datetime.timedelta(minutes=min)
63         ).isoformat()
64
65         if db.atualizarDatasPedido(int(pid), pago_em, previsto_para):
66             return 200, "Pagamento realizado com sucesso", {"pedido": pedido.__dict__}
67         else:
68             db.mudarEstadoPedido(int(pid), "PENDENTE")
69             return 0, "Falha ao atualizar datas do pedido", {}
70
71     return 0, "Falha ao realizar pagamento", {}
72

```

Figura 74 - Pagamento e envio de pedido

```

74
75 def get_pedido(dados, idCliente):
76     pid = dados.get("idPedido")
77
78     pedido = db.getPedido(int(pid))
79     idVendedor = db.getLoja(idLoja=db.getServico(pedido.idServico, pegar_apagado=True).idLoja).idCliente
80
81
82     if not pedido:
83         return 0, "Pedido não encontrado", {}
84
85     if int(pedido.idCliente) != idCliente and idVendedor != idCliente:
86         return 0, "Usuário não autorizado", {}
87
88     verificar_entrega(pedido)
89     # formato = "%Y-%m-%dT%H:%M:%S.%f"
90
91
92     return 200, "Pedido recuperado", {"pedido": pedido.__dict__}
93

```

Figura 75 - Consulta de pedido específico (cliente ou vendedor)

```

94  def get_pedidos(idCliente):
95      lista = db.getPedidos(idCliente)
96      return 200, "Pedidos do cliente", {"pedidos": lista}
97
98  def get_pedidos_minha_loja(idCliente):
99      lista = db.getPedidosLoja(idCliente)
100     return 200, "Pedidos da loja", {"pedidos": lista}
101
102  def cancelar_pedido(dados, idCliente):
103      pid = dados.get("idPedido")
104      pedido = db.getPedido(pid)
105      if not pedido:
106          return 0, "Pedido não encontrado", {}
107      if pedido.idCliente != idCliente:
108          return 0, "Usuário não autorizado", {}
109      ok = db.delPedido(pid)
110      if not ok:
111          return 0, "Falha ao cancelar pedido", {}
112      return 200, "Pedido cancelado com sucesso", {}
113

```

Figura 76 - Consultas e cancelamento de pedidos

As funções relacionadas aos anúncios de serviços permitem aos vendedores (donos de loja) gerenciar os serviços mágicos que oferecem no marketplace. A função **criar_anuncio** registra um novo serviço vinculado à loja do cliente autenticado. Já **get_servico** permite recuperar um serviço específico por seu ID, enquanto **get_categoria** retorna todas as categorias disponíveis. A visibilidade dos anúncios pode ser controlada com **mudar_estado_servico**, ocultando ou exibindo serviços na vitrine virtual. Vendedores podem ainda remover seus serviços com **deletar_servico** ou editar seus detalhes com **editar_servico**. Por fim, **get_catalogo** possibilita a listagem de serviços por categorias, loja e paginação, compondo o catálogo consultado pelos clientes.

```

6  def criar_anuncio(dados, idCliente):
7      try:
8          idLoja = db.getLoja(idCliente=idCliente).idLoja
9
10         servico = Servico(
11             nome_servico=dados["nome_servico"],
12             descricao_servico=dados["descricao_servico"],
13             categoria=dados["categoria"],
14             tipo_pagamento=dados["tipo_pagamento"],
15             quantidade=dados["quantidade"],
16             idLoja=idLoja,
17         )
18
19         id_servico = db.addServico(servico)
20
21         if id_servico is None:
22             return 0, "Erro ao criar o serviço: não foi possível inserir no banco", {}
23
24         servico.idServico = id_servico
25
26         return 200, "Serviço criado com sucesso", {"servico": servico.__dict__}
27
28     except Exception as e:
29         return 0, f"Erro ao criar serviço: {str(e)}", {}
30
31

```

Figura 77 - Criação de novo serviço (anúncio)

```

32  def get_servico(dados):
33      try:
34          idServico = dados["idServico"]
35          servico = db.getServico(idServico)
36          if servico:
37              return 200, "Serviço encontrado com sucesso", {"servico": servico.__dict__}
38          else:
39              return 0, "Serviço não encontrado", {}
40      except Exception as e:
41          return 0, f"Erro ao buscar serviço: {e}", {}
42
43
44  def get_categoria():
45      return 200, "Categorias mostradas com sucesso", {"categorias": categorias}
46
47

```

Figura 78 - Consulta de serviço e listagem de categorias

```

47
48 def mudar_estado_servico(dados, estado):
49     try:
50         idServico = dados["idServico"]
51         servico = db.getServico(idServico)
52         if servico:
53             db.mudarEstadoServico(idServico, estado)
54             servico.esta_visivel = estado
55             return (
56                 200,
57                 "Estado do serviço atualizado com sucesso",
58                 {"servico": servico.__dict__},
59             )
60         else:
61             return 0, "Serviço não encontrado", {}
62     except Exception as e:
63         return 0, f"Erro ao mudar estado do serviço: {e}", {}
64
65

```

Figura 79 - Alteração de visibilidade do serviço

```

66 def deletar_servico(dados, idCliente):
67     try:
68         idServico = dados["idServico"]
69         servico = db.getServico(idServico)
70         if not servico:
71             return 0, "Serviço não encontrado", {}
72         if servico.idLoja != db.getLoja(idCliente).idLoja:
73             return 0, "Usuário não autorizado a deletar este serviço", {}
74
75         db.delServico(servico.idServico)
76         return 200, "Serviço deletado com sucesso", {}
77     except Exception as e:
78         return 0, f"Erro ao deletar serviço: {e}", {}
79
80

```

Figura 80 - Remoção de serviço pela loja (forma lógica)

```

81 def editar_servico(dados, idCliente):
82     try:
83         idServico = dados["idServico"]
84         servico = db.getServico(idServico)
85         if not servico:
86             return 0, "Serviço não encontrado", {}
87         if servico.idLoja != db.getLoja(idCliente).idLoja:
88             return 0, "Usuário não autorizado a editar este serviço", {}
89
90         servico.nome_servico = dados["nome_servico"]
91         servico.descricao_servico = dados["descricao_servico"]
92         servico.categoria = dados["categoria"]
93         servico.tipo_pagamento = dados["tipo_pagamento"]
94         servico.quantidade = dados["quantidade"]
95
96         db.editarServico(servico)
97
98         return 200, "Serviço editado com sucesso", {"servico": servico.__dict__}
99     except Exception as e:
100         return 0, f"Erro ao editar serviço: {e}", {}
101

```

Figura 81 - Edição de dados de serviço

```

102
103 def get_catalogo(dados):
104     try:
105         servicos = db.getServicos(
106             categorias=dados["categorias"],
107             idLoja=dados["idLoja"],
108             cont_pages=int(dados["pages"]),
109         )
110         if not servicos:
111             return 200, "Nenhum serviço encontrado", {"servicos": []}
112         return 200, "Serviços encontrados com sucesso", {"servicos": servicos}
113
114     except Exception as e:
115         return 0, f"Erro ao buscar serviços: {str(e)}", {}
116

```

Figura 82 - Catálogo de serviços filtrado por categoria e loja

4. Resultados

O sistema desenvolvido resultou em uma interface gráfica completamente integrada à arquitetura cliente-servidor, proporcionando uma experiência de uso fluida, responsiva e intuitiva.

O fluxo de autenticação e controle de permissões foi implementado de forma rigorosa, assegurando que apenas usuários autenticados possam acessar funcionalidades sensíveis ou restritas.

A seguir, são descritas as principais páginas implementadas:

Página Inicial (Home)

- **Descrição:** Tela de boas-vindas ao sistema, com o logo do MarketPlace.
- **Funcionalidades:** Apresentação do MarketPlace e acesso às principais seções por meio da navegação lateral. (Figura 83).

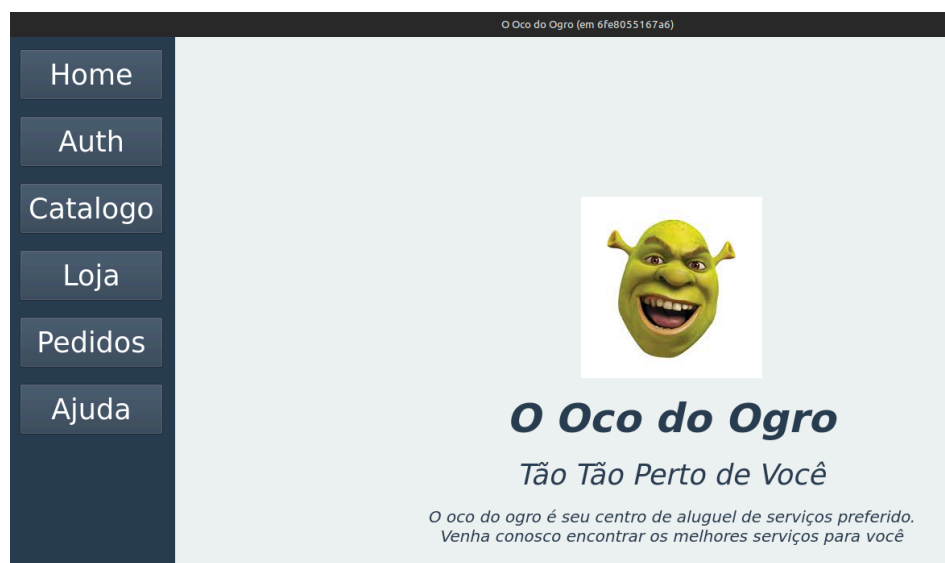


Figura 83 - Página Inicial.

Autenticação de Usuário

- **Login e Cadastro:** Disponíveis na seção "Auth" da barra lateral, permitem que usuários se autenticem ou criem novas contas. (Figura 84).



Figura 84 - Login e Cadastro.

- **Verificação de Acesso:** O sistema realiza validação automática de autenticação ao tentar acessar qualquer página restrita. Usuários não autenticados são redirecionados para a área de login. (Figura 85).

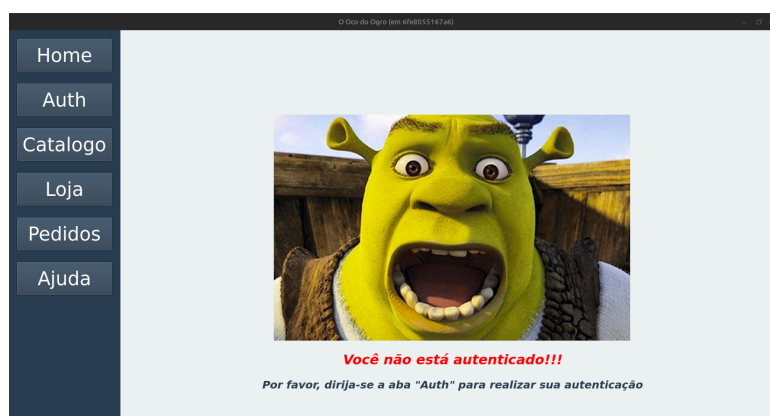


Figura 85 - Verificação de Acesso.

- **Logout:** Após login, a opção "Logout" torna-se disponível na seção "Auth", permitindo o encerramento seguro da sessão. (Figura 86).

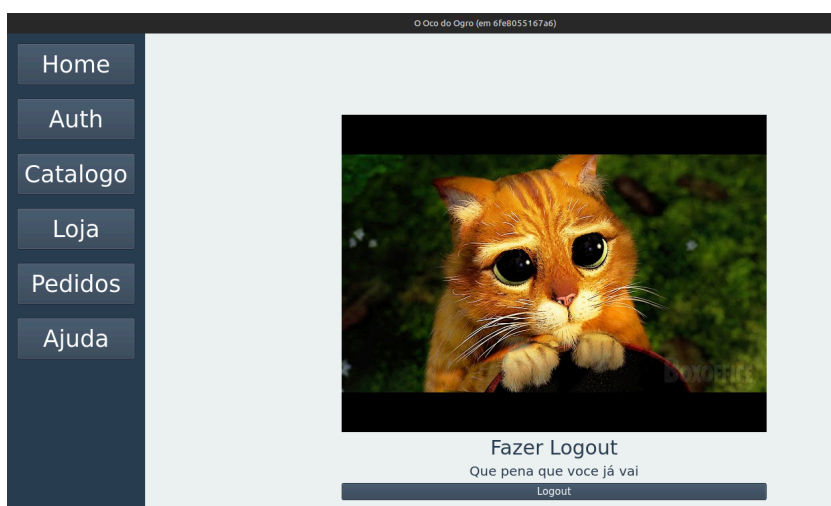


Figura 86 - Logout.

Gerenciamento de Lojas

- **Criação de Loja:** Usuários autenticados podem criar uma loja ao acessar a área correspondente pela primeira vez. (Figura 87).

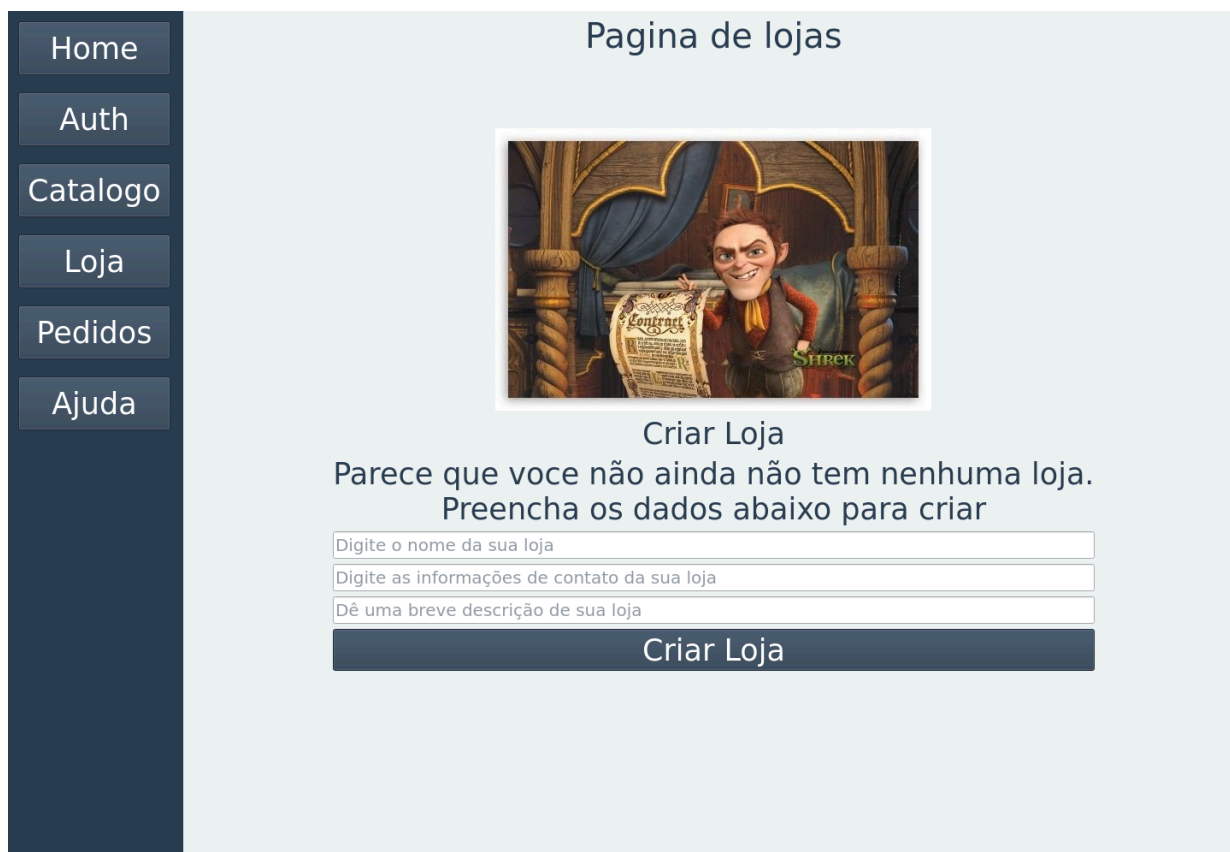


Figura 87 - Criar Loja.

- **Área da Loja:** A criação de uma loja habilita o usuário ao acesso a Área de sua Loja, que oferece as funcionalidades de cadastrar e gerenciar serviços.(Figura 88).



Figura 88 - Área Loja.

Catálogo de Produtos e Serviços

- **Visualização:** Exibe os produtos e serviços disponíveis no sistema.
- **Filtros e Pesquisa:** Permite busca refinada por meio de filtros e critérios de pesquisa.

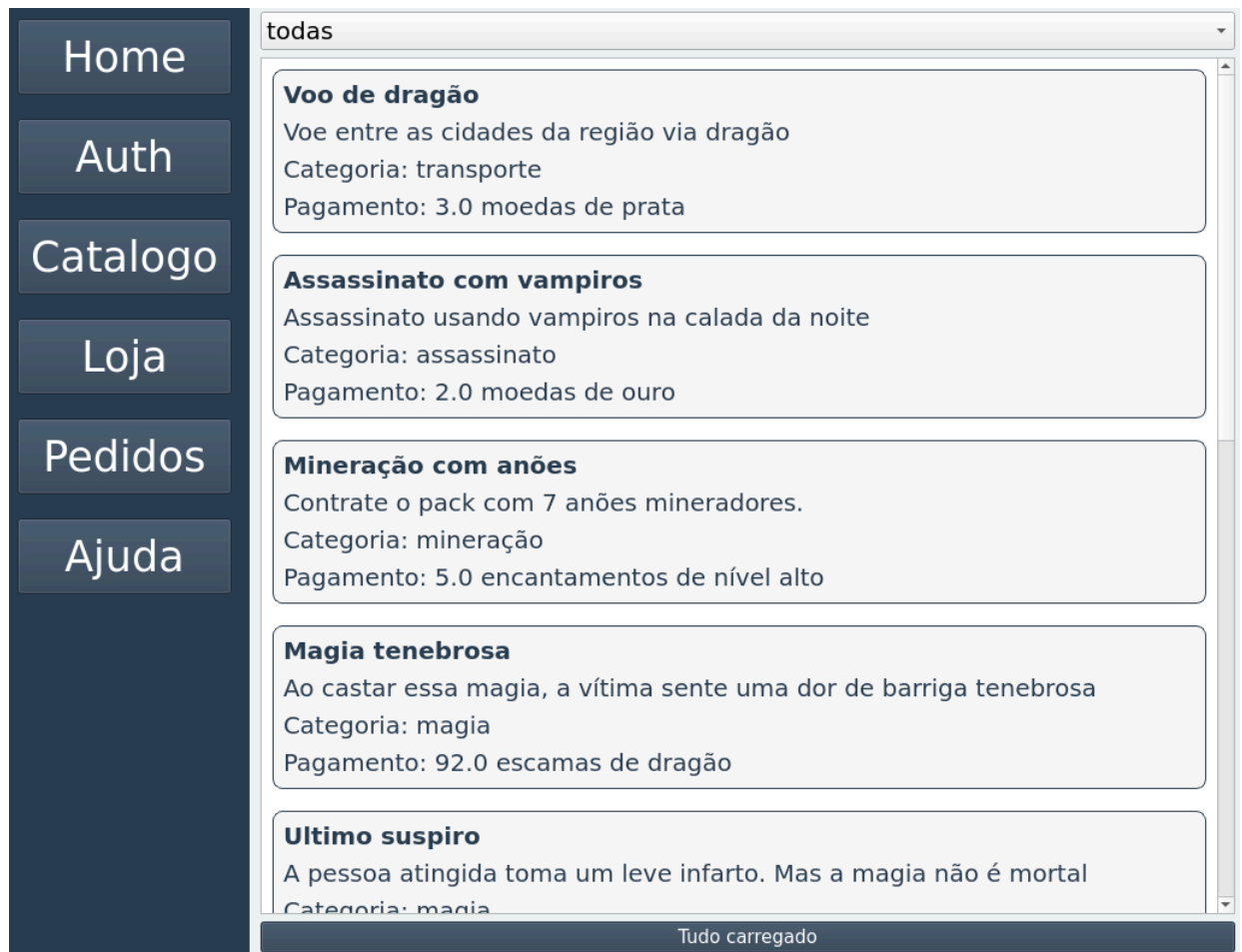


Figura 89 - Catálogo.

Pedidos Feitos pelo Cliente

- **Funcionalidades:**
 - Realização de pedidos;
 - Acompanhamento de status e informações dos pedidos;
 - Processamento de pagamentos;
 - Cancelamento de pedidos;



Figura 90 - Área de Meus Pedidos.

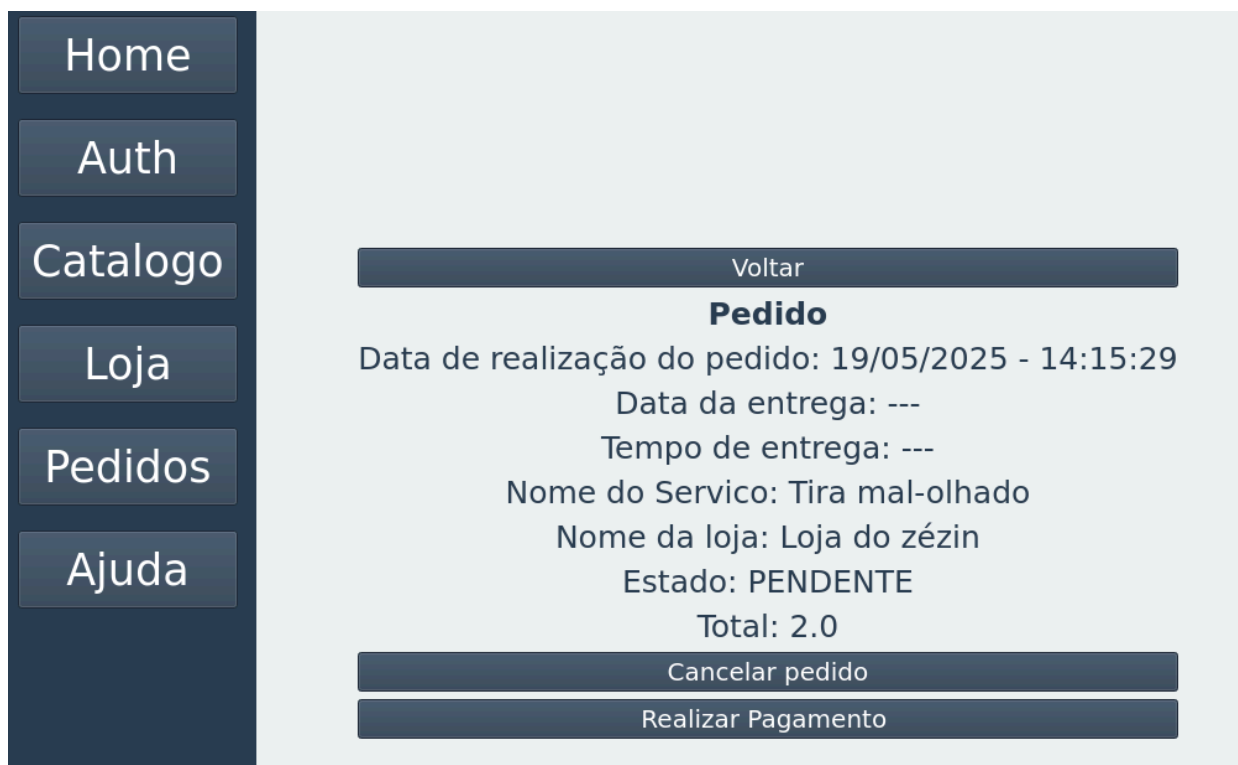


Figura 91 - Estado do meu pedido.



Figura 92 - Pagamento.



Figura 93 - Estado pós pagamento.

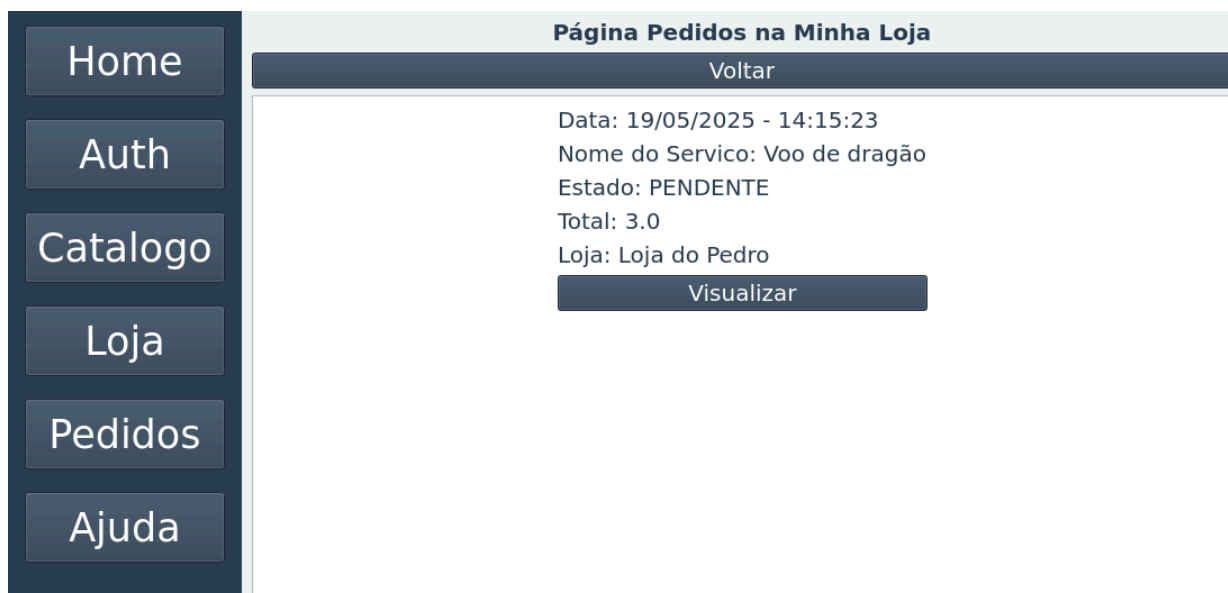


Figura 94 - Pedidos feitos na minha loja.

Anúncios

- **Gerenciamento de Anúncios:** Usuários com loja têm acesso às seguintes operações:
 - Criação e edição de anúncios; (Figura 88).

- Ocultação e exibição de anúncios; (Figura 89).
- Exclusão de anúncios.(Figura 89).

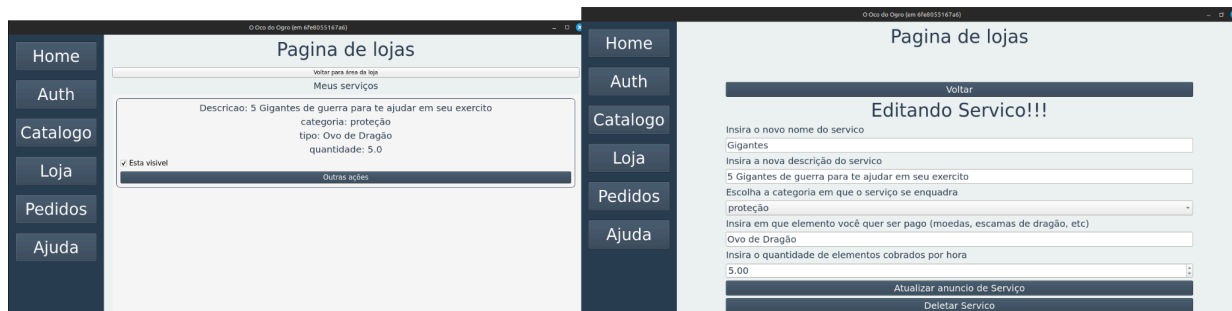


Figura 95 - Ocultar e Exibir anúncios à esquerda. Editar e Excluir anúncios á direita.

Suporte

- **Página de Ajuda:** Oferece instruções e respostas a dúvidas frequentes. (Figura 90).



Figura 96 - Ocultar e Exibir anúncios à esquerda. Editar e Excluir anúncios á direita.

5. Conclusão

O sistema distribuído foi implementado com sucesso, atendendo aos requisitos propostos, incluindo uso de sockets em Python 3.12, banco de dados SQLite com

persistência em arquivo, e execução padronizada via Docker.

As principais dificuldades encontradas foram a familiarização com a API de sockets, a definição do formato das mensagens e a conciliação do tempo disponível para o desenvolvimento. Por se tratar de um sistema com múltiplas funcionalidades, o prazo curto exigiu organização e priorização de tarefas, o que limitou a adição de melhorias extras.

Com apoio do material da disciplina e uma divisão clara dos módulos (cliente, servidor e interface), conseguimos superar os desafios e garantir uma estrutura funcional e modular. A utilização de threads no servidor permitiu lidar com múltiplas conexões simultâneas de forma eficiente. Sempre que necessário, foram aplicados mecanismos de sincronização para evitar conflitos no acesso ao banco de dados.

O projeto não só cumpriu seu papel pedagógico, ajudando a consolidar conceitos teóricos passados, como também resultou em um sistema funcional e confiável. Acreditamos que o aprendizado obtido será aplicável em projetos futuros que envolvam comunicação entre processos e arquitetura distribuída.

6. Referências

Para versionar o projeto foi utilizado o Github [1], e o Git.

[1] Github. Disponível em: <https://github.com/GuilhermeZorzal/TP_Distribuidos>

Último acesso em: 18 de maio de 2025.

[2] SILVA, Thais Regina de Moura Braga. *CCF 355 – Sistemas Distribuídos e Paralelos*. [Material didático]. Universidade Federal de Viçosa, 2025.

[3] PYTHON SOFTWARE FOUNDATION. *socket — Low-level networking interface*.

Disponível em: <https://docs.python.org/3/library/socket.html>. Último acesso em: 16 maio 2025.

[4] THE SQLITE CONSORTIUM. *SQLite Documentation*. Disponível em: <https://www.sqlite.org/docs.html>. Último acesso em: 16 maio 2025.

[5] QT COMPANY. *PyQt6 Documentation*. Disponível em: <https://doc.qt.io/qtforpython-6/>. Último acesso em: 16 maio 2025.

